数据结构复习

By:Sukuna (CSXJ1902) With Markdown

绪论

1.1 基本概念与术语
数据、数据元素、数据项、数据对象、数据结构
数据存储结构（顺序存储结构、链式存储结构）
数据类型（原子类型、结构类型）
抽象数据类型

数据：是能输入到计算机中并能被计算机程序处理的符号的总称。
数据元素：是数据的基本单位，它在计算机处理和程序设计中通常作为一个整体进行考虑和处理。一个数据元素可由若干数据项组成。
数据对象：是具有相同特征的数据元素的集合，是数据的一个子集。
数据结构：是数据元素的组织形式，或数据元素相互之间存在一种或多种特定关系的集合。
数据的存储结构：是数据的逻辑结构在计算机内存中的存储方式，又称物理结构。 数据类型：是一组具有相同性质的操作对象以及该组操作对象上的运算方法的集合。
抽象数据类型：是指一个数学模型以及在该模型上定义的一套运算规则的集合。

1.2 算法的时间复杂度
语句频度,记作f(n)：基本语句执行次数总和
时间复杂度记作T(n)， T(n)=O(f(n))

频度的计算:怎么算得知道

线性表

2.1 线性表的定义
记作: L=(a1,a2,...,an)，特征:一对一的计算

2.2 线性表的顺序表示（顺序存储结构）
地址计算:$b+(n-1)p$
插入、删除算法并移动元素分析

结构表示

```
//动态表示
#define LIST_INIT_SIZE 100
 #define LISTINCREMENT 10
typedef struct
   { ElemType *elem; //存储空间基地址
     int length;     //表长
     int listsize;   //当前分配的存储容量
                     // (以sizeof(ElemType)为单位
    } SqList;
//静态表示就微调
```

插入算法(静态插入)

```
Status Insert2(SqList &L,int i,ElemType e)
   { if (i<1||i>L.length+1) return ERROR      //i值不合法
     if (L.length>=maxleng) return OVERFLOW  //溢出
     for (j=L.length-1; j>=i-1; j--; )
         L.elem[j+1]=L.elem[j];        //向后移动元素
     L.elem[i-1]=e;                    //插入新元素
     L.length++;                       //长度变量增1
     return OK                          //插入成功
    }
```

动态插入

```
int Insert3(SqList &L,int i, ElemType e)
  {int j;
   if (i<1 || i>L.length+1)      //i的合法取值为1至n+1
          return ERROR;
   if (L.length>=L.listsize)     /*溢出时扩充*/
    {
     ElemType *newbase;
     newbase=(ElemType *) realloc(L.elem,
    (L.listsize+LISTINCREMENT)*sizeof(ElemType));
    if (newbase==NULL) return OVERFLOW;    //扩充失败
     L.elem=newbase;
     L.listsize+=LISTINCREMENT;
    }
//注意扩充的方法
 //向后移动元素，空出第i个元素的分量elem[i-1]
for(j=L.length-1;j>=i-1;j--)
  L.elem[j+1]=L.elem[j];
L.elem[i-1]=e;                     /*新元素插入*/
L.length++;                        /*线性表长度加1*/
return OK;
}
```

插入:移动元素的平均值为$\frac{n}{2}$,删除:$\frac{n-1}{2}$

删除操作:从前往后移动,插入操作:从后往前移动

评价:

1.优点:
(1)是一种随机存取结构，存取任何元素的时间是一个常数，速度快；
(2)结构简单，逻辑上相邻的元素在物理上也是相邻的；
(3)不使用指针，节省存储空间。
2.缺点:
(1)插入和删除元素要移动大量元素，消耗大量时间；
(2)需要一个连续的存储空间；
(3)插入元素可能发生“溢出”；
(4)自由区中的存储空间不能被其它数据占用(共享)。

2.3 线性表的链式存储结构
单链表：插入、删除算法、合并算法及其他
循环链表：插入、删除元素指针变化
双向链表：插入、删除元素指针变化

双向循环链表：插入、删除元素指针变化

2.3.1链表

带头结点和不带头结点
带头结点的作用:需要额外开辟一个空间,但是插入和删除首元素时不用进行特殊处理

静态链表:用数组模拟,数组里面有data域和下一个元素的编号,head->0,尾元素指向0

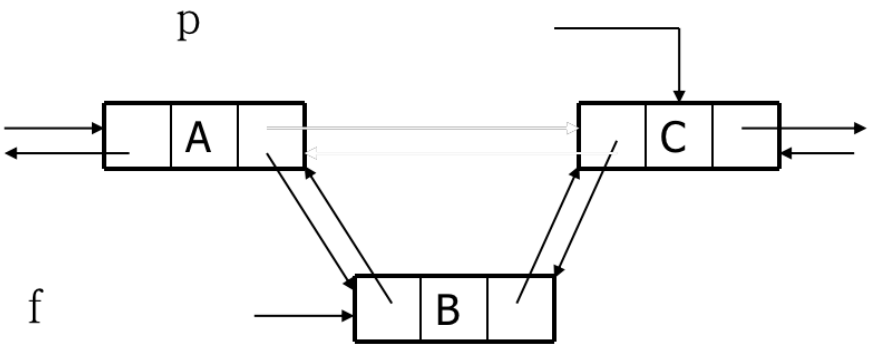
```
typedef struct node
      { ElemType data;      //data为抽象元素类型
        struct node *next;  //next为指针类型
       } Lnode, *Linklist;
```

2.3.2 循环链表:
尾指针:有头:指向表头
　　无头:指向第一个元素
head:设置任意一个指
只需要设一个尾指针就可以满足,可以找到链表的任何一个值

2.3.3 双向链表

首结点前驱是空,尾结点后继是空
表头结点前驱是尾结点,后继指头结点
会考一些操作

2.3.4 双向循环链表:没有哪一个指针域是空



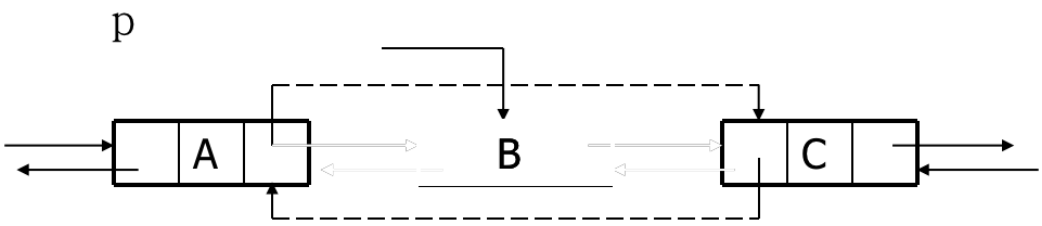
执行:

① f->prior=p->prior;　//结点B的prior指向结点A

② f->next=p;　//结点B的next指向结点C

③ p->prior->next=f;　//结点A的next指向结点B

④ p->prior=f;　//结点C的prior指向结点B



执行：

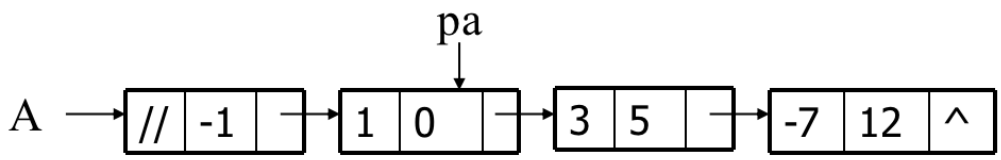
```
p->prior->next=p->next;     //结点A的next指向结点C
p->next->prior=p->prior;    //结点C的prior指向结点A
free(p);                    //释放结点B占有的空间
```

会考一些操作

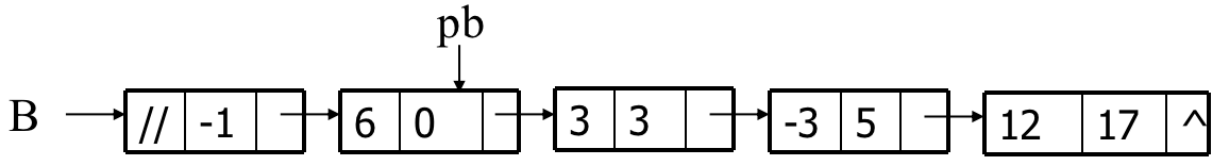
2.4 一元多项式的表示及相加



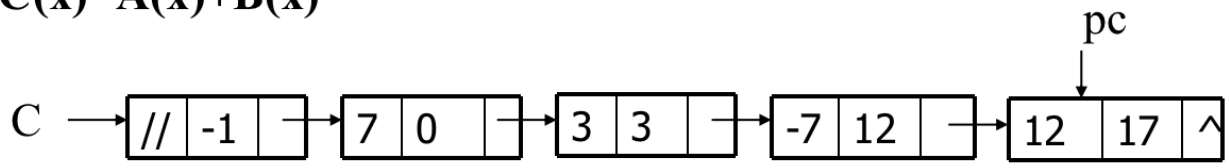
$A_{12}(x)=1+3x^5-7x^{12}$



$B_{17}(x)=6+3x^3-3x^5+12x^{17}$



$C(x)=A(x)+B(x)$



栈和队列

3.1 栈的定义

栈：限定在表尾作插入、删除操作的线性表。

出栈、入栈的特征；一个输入序列通过栈有不同的输出序列

特征:先入后出,多少个不同序列:第二斯特林数

3.2 栈的顺序存储
如何分配存储空间
如何设置进栈和出栈的标志top
分析满栈与空栈的条件
进栈、出栈算法

存储形式:

```
#define STACK_INIT_SIZE  100
#define STACKINCREMENT   10
typedef struct
 { ElemType *base;     //指向栈元素空间
   int top;            //顶指针
   int stacksize;       //当前分配的栈空间大小
} SqStack;             // SqStack为结构类型
```

top:几个定义法:a_n和a_n的上一个单元:空栈时分别对应-1和0,对应有不同溢出判断法:一个是top指向maxlength-1,一个指向maxlength,对应的进栈、出栈算法也不同,就是top++的位置不一样

3.3 栈的链式存储
进栈算法
退栈算法

存储形式:

```
struct node
    { ElemType data;       //data为抽象元素类型
      struct node *next;   //next为指针类型
     } *top=NULL;          //初始化,置top为空
```

算法

```
  p->next=top;                  //插入新结点
  top=p;                        //top指向新结点

  p=top;                        //p指向原栈的顶结点
  top=top->next;              //删除原栈的顶结点
```

3.4 栈的应用算法
进制转换
括号匹配
表达式求值

3.5 队列的定义:队列----只允许在表的一端删除元素,在另一端插入元素的线性表。
 出队、进队的特征:在队首取出,在队尾插入

双向队列:中间值不能动
输入受限:只允许在表的一端插入、在两端删除元素的线性表
输出受限:只许在表的两端插入、在一端删除元素的线性表

3.6 链式队列（单链表）
 区分带表头与不带表头
 插入、删除算法（入队、出队）

存储形式:

```
// (1)存放元素的结点类型
   typedef struct Qnode
   { ElemType data;         //data为抽象元素类型
     struct Qnode *next;    //next为指针类型
    }Qnode, *QueuePtr;      //结点类型, 指针类型
//其中: Qnode----结点类型
//      QueuePtr----指向Qnode的指针类型

// (2)由头、尾指针组成的结点类型
   typedef struct
   { Qnode *front;  //头指针
     Qnode *rear;   //尾指针
    }LinkQueue;     //链式队列类型
```

入队:

```
   p->next=NULL;                //为队尾结点
   Q->rear->next=p;                 //插入新结点
   Q->rear=p;                       //修改尾指针
```

出队

```
   p = Q.front->next;             //P指向队头结点
   Q.front->next=p->next;      //删除队头结点
   if (Q.rear==p)              //若原队列只有1个结点
      Q.rear=Q.front;         //修改尾指针
   free(p);                    //释放被删除结点的空间
```

3.7 队列的顺序存储
 如何用下标判断满队列从而防止溢出
防止溢出:循环队列;防止假溢出:留一个不存数据
入队算法、出队算法

队列定义

```
#define MAXLENG 100
Typedef struct
  {
  ElemType elem[MAXLENG];
  int    front,rear;
  } SeQueue;
```

溢出判断:$(Q.rear+1)\%MAXLENG = Q.front$
下溢判断:$Q.front == Q.rear$
算队列长的方法:$(Q.rearQ.front + maxlength)\%maxlength$

串

4.1 串的定义
 相关术语:串值、串长、空串、子串等

4.2 串的基本操作与函数
4.3 串的顺序存储
字符堆:指针指向一个地址(第一个字母的地址),还有一个元素存长度
'\0'C语言
a[0]存指定的串长 Pascal

4.4 串的链式存储

链式堆的存储密度
(1)多字符和单字符
(2)计算方式:$\frac{\text{有效字符}}{\text{开辟的总空间}}$,开辟的总空间=字符数+指针域

4.5 KMP

数组与广义表

5.1 数组的定义
A= (a1,a2,...,an) 一维数组，多维数组
5.2 数组的顺序存储结构
如何计算元素的存储地址（一维、多维）

例子:三维数组a[1..p,1..m,1..n],假定无0页0行0列
(1)以行序为主序，$a[k][i][j]$的地址为
$Loc(k,i,j)=Loc(1,1,1)+(m*n*(k-1)+n(i-1)+j-1)*s=h+(m*n*(k-1)+n(i-1)+j-1)*s$
$1\le k\le p,1\le i\le m,1\le j\le n$

(2)以列序为主序，$a[k][i][j]$的地址为
$Loc(k,i,j)=Loc(1,1,1)+(p*m*(j-1)+p*(i-1)+k-1)*s=h+(p*m*(j-1)+p*(i-1)+k-1)*s$

$1\le k\le p,1\le i\le m,1\le j\le n$

如果有0行0列:所有-1去掉

5.3 矩阵的压缩存储（对称、对角、稀疏）

(1)对称矩阵

a_{11}	a_{21}	a_{22}	a_{31}	...	a_{i1}	...	a_{ij}	...	a_{ii}	...	a_{n1}	...	a_{nn}

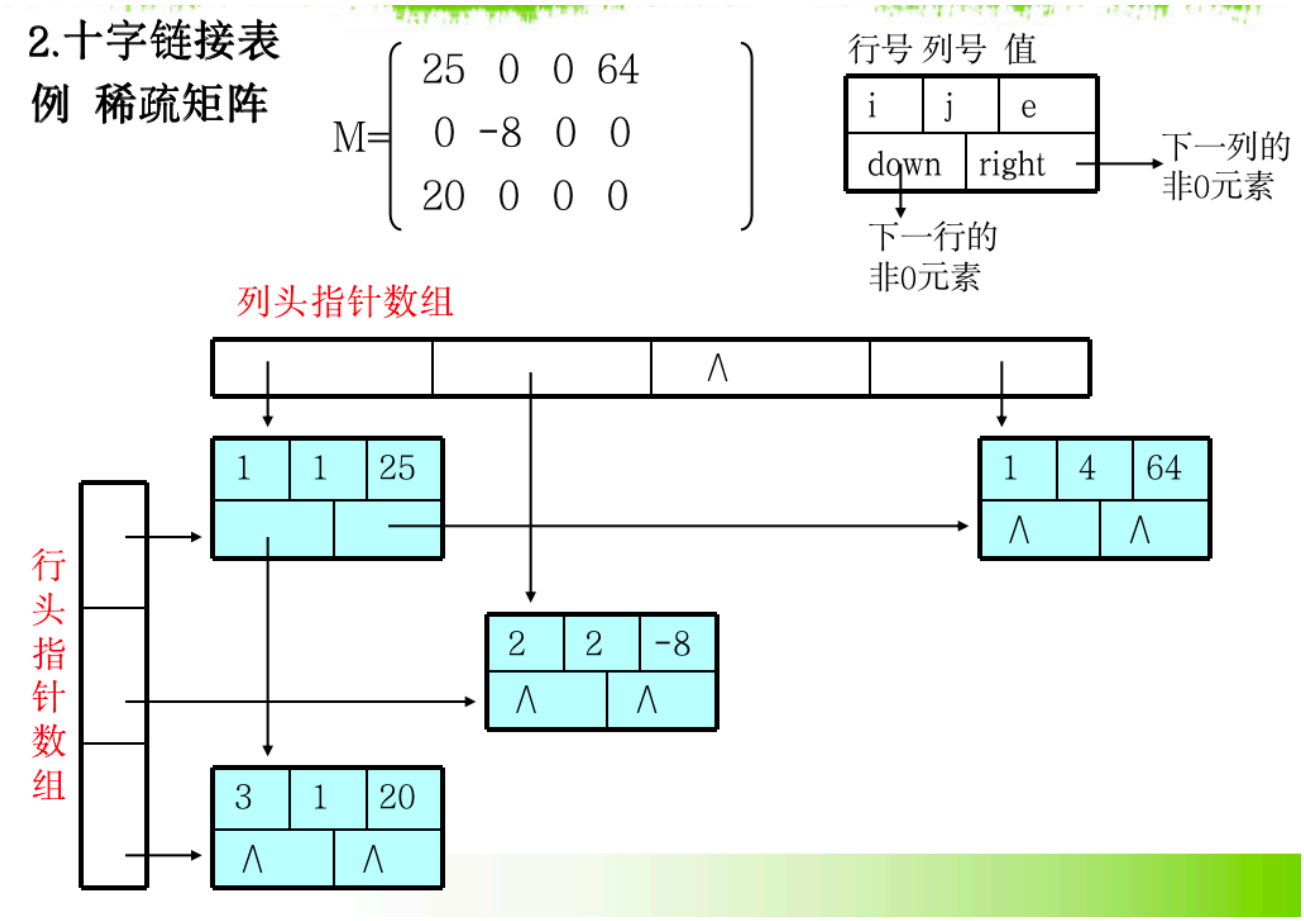
$$k(i,j)=\begin{cases} i(i-1)/2+j & i\geqslant j \\ j(j-1)/2+i & i<j \end{cases}$$

(2)三对角矩阵

a_{11}	a_{12}	a_{21}	a_{22}	a_{23}	a_{32}	...	a_{ij}	...	a_{nn-1}	a_{nn}
k=1	2	3	4	5	6	...	?	...		3n-2

$k=(3*(i-1)-1)+(j-i+2)=2(i-1)+j$

3.稀疏矩阵
(1)三元组表:存行列值和元素值
(2)十字链表



5.3 广义表的定义与相关术语
LS=(e1,e2,...,en)，表头、表尾
5.4 广义表的相关操作
求长度、表头、表尾、第i个元素、深度、插入

操作集锦

```
1.求长度：Leng(LS)
 a=( )              Leng(A)=0
 G=(a,G)            Leng(G)=2
 H=(( ),(( ),( )))  Leng(H)=2
 F=(A,B,C,d)        Leng(F)=4

2.求表头： Head(LS)
   G=(a,G)              Head(G)=a
   E=((a,b),c,(d,e))    Head(E)=(a,b)
3.求表尾： Tail(LS)
   G=(a,G)              Tail(G)=(G)=((a,G))
   E=((a,b),c,(d,e))    Tail(E)=(c,(d,e))

4.求第i个元素：GetElem(LS,i)=ei    1≤i≤n
   I=((a,b),c,( ),(d))
   GetElem(I,1)=(a,b)     Get(I,2)=c
   GetElem(E,3)=( )       Get(I,4)=(d)
```

```
5.求深度:  Depth(LS)---LS所含括号的层数
 (1) A=( )                Depth(A)=1
 (2) E=((a,b),c,(d,e))    Depth(E)=2
 (3) H=(( ),(( ),( )))    Depth(H)=3
```

广义表:加递归,元素可以是单独的值也可以是新的广义表

结构位置表示:a,b,c,d 都表明好
(1)广义表的图型表示----树型结构
约定 □----单元素/原子
　　○----列表,若有表名,附表名：对应（）
(2)存储结构



(3)表头:第一个元素:可元素,可表
(4)表尾:剩下的元素都是表尾,一定是广义表

5.5 广义表的链式存储结构及图示

树

6.1 树的定义及相关概念
　度、叶子、分枝结点、层、深度、兄弟、有序树、无序树、森林

度:结点子树的数目
树的度:各结点度的最大值
n度树:度为n的树
分枝结点(非终端结点,非叶子):度不为0的结点
叶子结点:度为0
层:规定树T的根的层为1，其余任一结点的层等于其双亲的层加1。
树的深度(depth,高度):树中各结点的层的最大值。
有序树:若任一结点的各棵子树，规定从左至右是有次序的,即不能互换位置,则称该树为有序树。
无序树: 若任一结点的各棵子树，规定从左至右是无次序的,即能互换位置,则称该树为无序树。

6.2 二叉树的定义及相关性质
　结点数与层数、深度、度的关系等（相关推导过程）

结点数:深度为k的二叉树最多有2^k-1个结点
$2^0+2^1+2^2+.....+2^{k-1}=2^k-1$
叶子的数目=度为2的结点数目+1

　满二叉树的定义、编号及相关性质
深度为k且有2^k-1个结点的二叉树。
n个结点的满二叉树的深度=$log_2(n+1)$,结点i的层号= $down(log_2 i)+1$ =$upper(log_2(i+1))$
结点i的左小孩是2i,　　2i≤n
结点i的右小孩是2i+1,　2i+1≤n
结点i的双亲是down(i/2);　　2≤i≤n

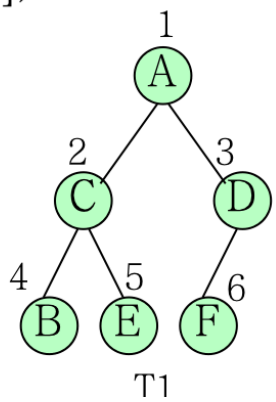
完全二叉树的定义及相关性质
深度为k的有n个结点的二叉树,当且仅当每一个结点
都与同深度的满二叉树中编号从1至n的结点一一对应,称
之为完全二叉树（其它教材称为“顺序二叉树”）

6.3 二叉树的顺序存储结构（利用编号）

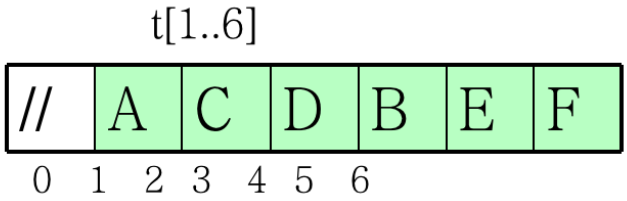
(1) n个结点的完全二叉树，使用一维数组：

例. ElemType tree[n+1];

char t[7];



T1



T1的顺序结构

- 元素(结点)t[i]的双亲是t[i/2], 2≤i≤n
- 元素(结点)t[i]的左小孩是t[2i], 2i≤n
- 元素(结点)t[i]的右小孩是t[2i+1], 2i+1≤n

6.4 二叉树的链式存储结构

二叉链表、三叉链表、静态链表、相关数据结构定义
遍历二叉树算法、线索二叉树算法、生成二叉树算法
中序遍历序列和前(或后)序序列确定唯一棵二叉树
求二叉树中结点的个数算法、查找二叉树某个结点的算法及其他算法

二叉链表

```
struct BiTNode              //结点类型
   { struct BiTNode * lchild; //左孩子指针
     ElemType data;           //抽象数据元素类型
     struct BiTNode * rchild; //右孩子指针
    } *root,*p;
```

三叉链表 有父结点

```
struct BiTNode
   { struct BiTNode *parent,*lchild,*rchild ;
     ElemType data;
    } *root,*p;
```

静态链表 存左右孩子的位置

```
struct bnode2
{ ElemType data;
  int lchild,rchild;
 }t[n+1];
```

中序遍历非递归算法
思路就是先找到最左的结点,把经过的根结点都保存了,后来左子树遍历完了就找上面根结点的右子树,再把右子树看成个树遍历,遍历完了就返回上一级(退栈),相当于更大的左子树访问完了

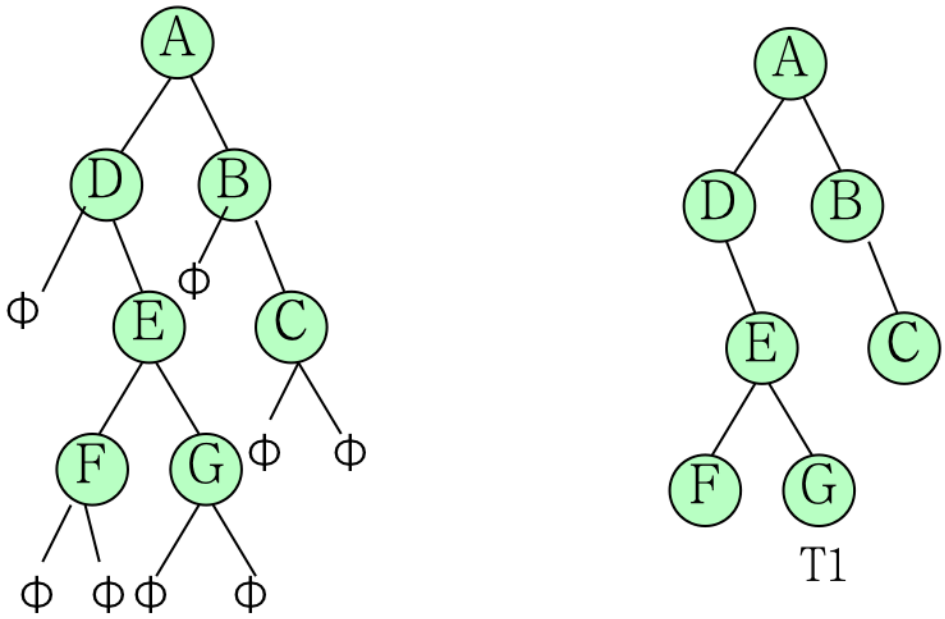
```
void Midorder(struct BiTNode *t)       //t为根指针
{ struct BiTNode *st[maxleng];//定义指针栈
  int top=0;                    //置空栈
  do{
    while(t)                //根指针t表示的为非空二叉树
    { if (top==maxleng) exit(OVERFLOW); //栈已满,退出
      st[top++]=t;              //根指针进栈
      t=t->lchild;              //t移向左子树
     }   //循环结束表示以栈顶元素的指向为
         //根结点的二叉树的左子树遍历结束
    if (top)                    //为非空栈
    { t=st[--top];              //弹出根指针
      printf("%c",t->data);     //访问根结点
      t=t->rchild;              //遍历右子树
     }
   } while(top||t);  //父结点未访问，或右子树未遍历
 }
```

先序遍历的非递归实现

```
void Preorder(struct BiTNode * t){
  struct BiTNode * St[MaxSize], *p;
  int top = 0;       //置空栈
  if (t! = NULL){
    St[top++] = t;
    while(top){
     p = St[--top]; printf("%c ", p->data);
     if(p->rchild != NULL)
        St[top++] = p->rchild;
     if(p->lchild != NULL)
        St[top++] = p->lchild;
    }
    printf("\n");
  }
}
```

通过带Φ的序列构建二叉树

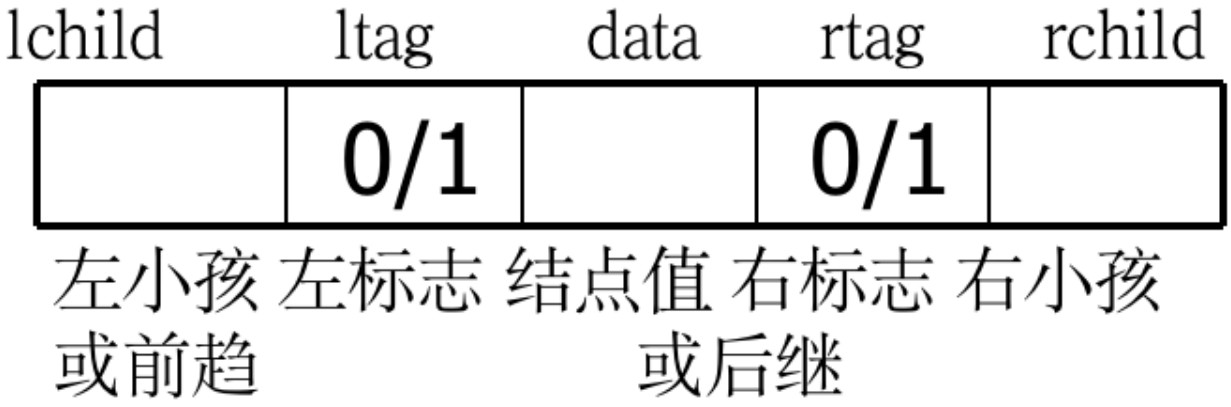
A D Φ E F Φ Φ G Φ Φ B Φ C Φ Φ



遇到Φ就返回上一层

线索二叉树:

二叉树一共有2n个指针域,占用了n-1和,剩下n+1个就会被利用,分为前序线索二叉树，中序线索二叉树， 后序线索二叉树



弹出根结点,如果结点有左孩子话,tag为0,没有的话记为1,注意保存一个pre,保存序列上一个结点的内容,总之很抽象,需要好好研读

遍历算法:

```
void creat_thread(struct BiTNode *t)
{ struct BiTNode *st[maxleng+1];  //指针栈
  int top=0;                      //置空栈
  struct BiTNode *pre=NULL;       //前驱结点指针
  do
  { while(t)         //根指针t表示的为非空二叉树
```

```
    { if (top==maxleng)
            exit(OVERFLOW);     //栈已满,退出
      st[top++]=t;              //根指针进栈
      t=t->lchild;              //t移向左子树
    }
    if (top)                                    //为非空栈
    { t=st[--top];                          //弹出根指针
      printf("%c",t->data);                //访问根结点
      if (t->lchild!=NULL)  t->ltag=0;     //左指针为孩子
      else   {
              t->ltag=1;t->lchild=pre; } //左指针为线索
      if (pre!=NULL)
          if (pre->rchild!=NULL)
                pre->rtag=0;            //右指针为孩子
          else {
                pre->rtag=1;            //右指针为线索
                pre->rchild=t;
                }
      pre=t;                            //pre与t保持前后
      t=t->rchild;              //遍历右子树
     }
   } while(top||t);
 pre->rtag=1;                   //最后一节点右标记线索
```

中序遍历序列和前(或后)序序列确定唯一棵二叉树

中:划分左右,前或者后:确定根结点

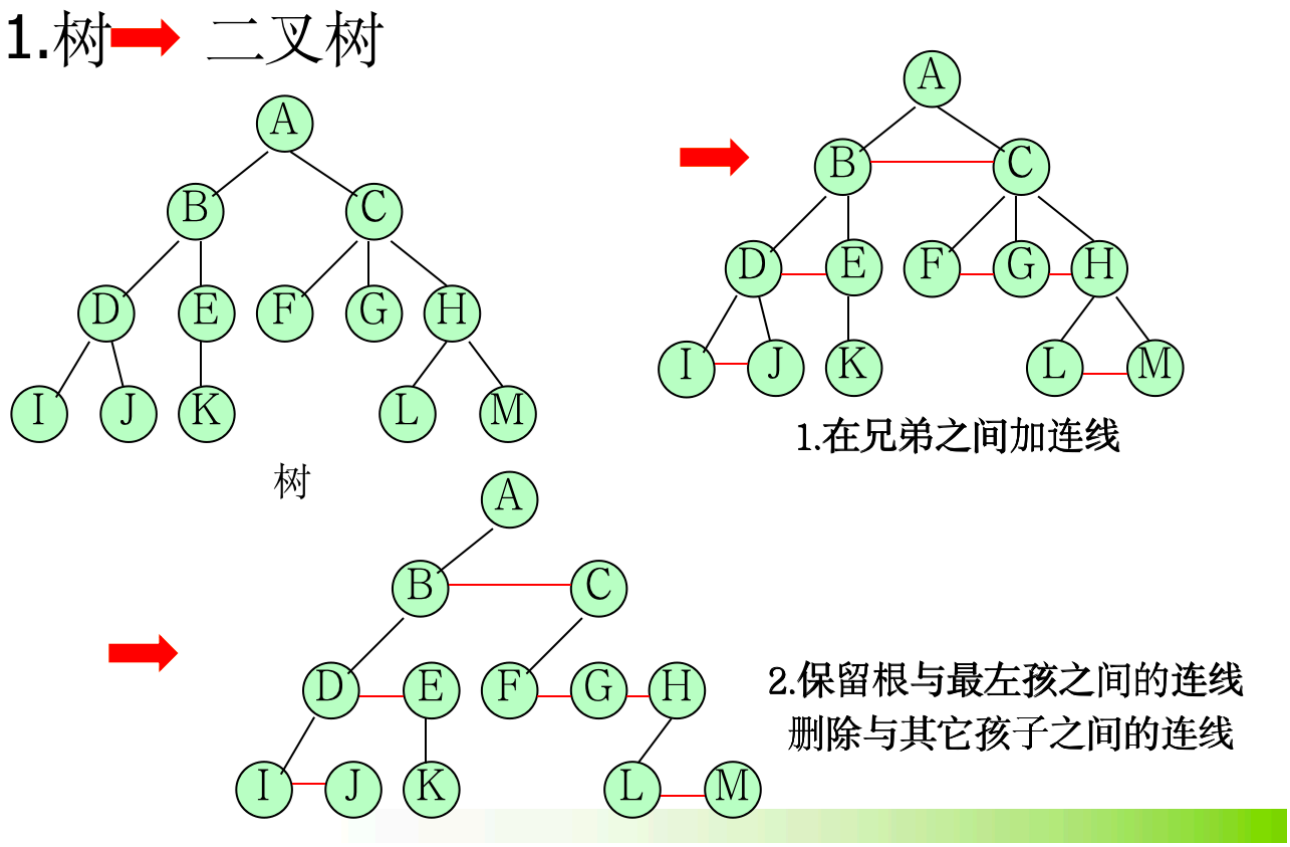
6.5 树的存储结构
双亲表示法、链接表示法、二叉链表表示法、单链表表示法、带双亲的孩子链表表示法
树和二叉树的转换
森林和二叉树的转换
树和森林的遍历（前序遍历、后序遍历）

构建形式:

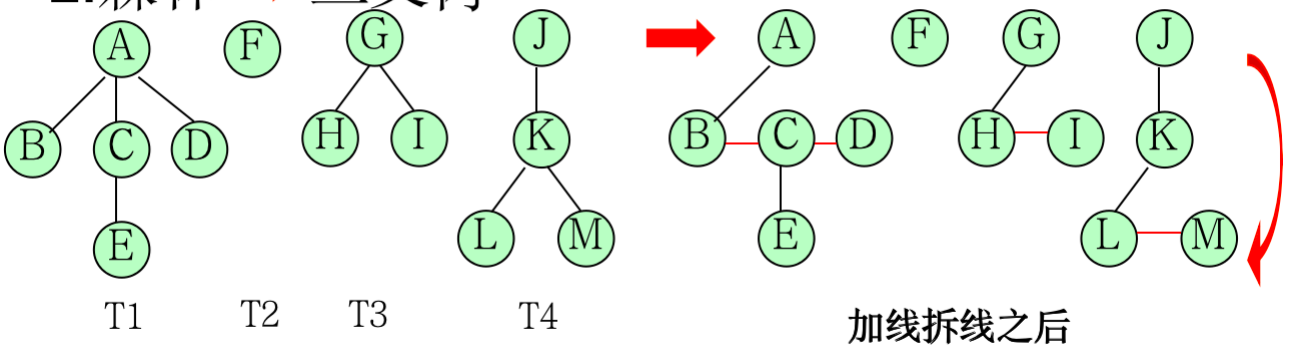
```
1.双亲表示法
struct snode
{ char data;
  int parent;
}t[maxleng+1};
2.链接表表示法:
(1)固定大小
struct snode
{ char data;
  struct snode* child1;
  struct snode* child2;
  struct snode* child3;
}t;
(2)不固定大小
struct snode
{ char data;
  int num;
```

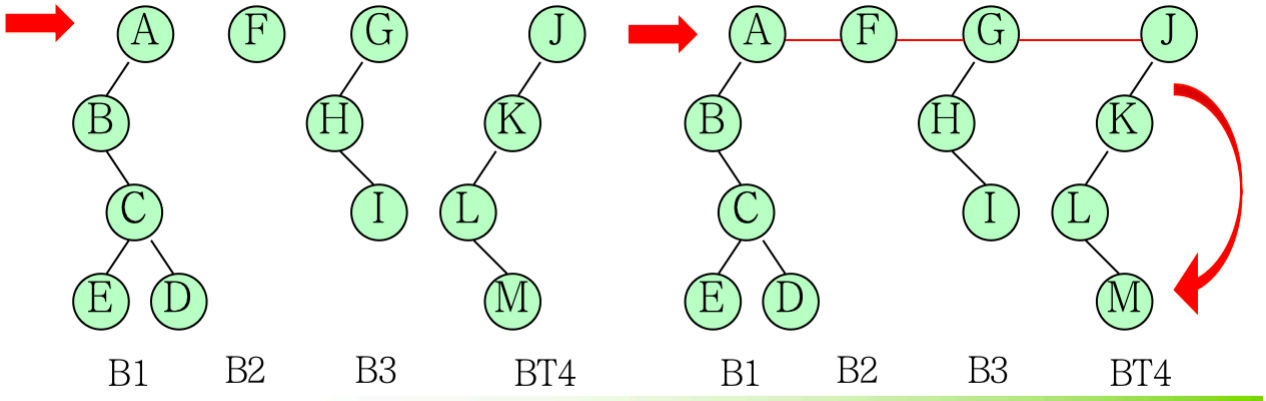
```
  struct snode* child1;
  struct snode* child2;
  struct snode* child3;
}t;
3.单链表表示法
struct snode
{ char data;
  struct edge* first;
}t[maxlength+1];
4.带双亲的孩子链表表示法：
struct snode
{ char data;
  int parent;
  struct edge* first;
}t[maxlength+1];
5.孩子兄弟表示法
struct snode
{ char data;
  struct snode* firstchild;
  struct snode* sibilings;
}t;
```

树与二叉树的转换



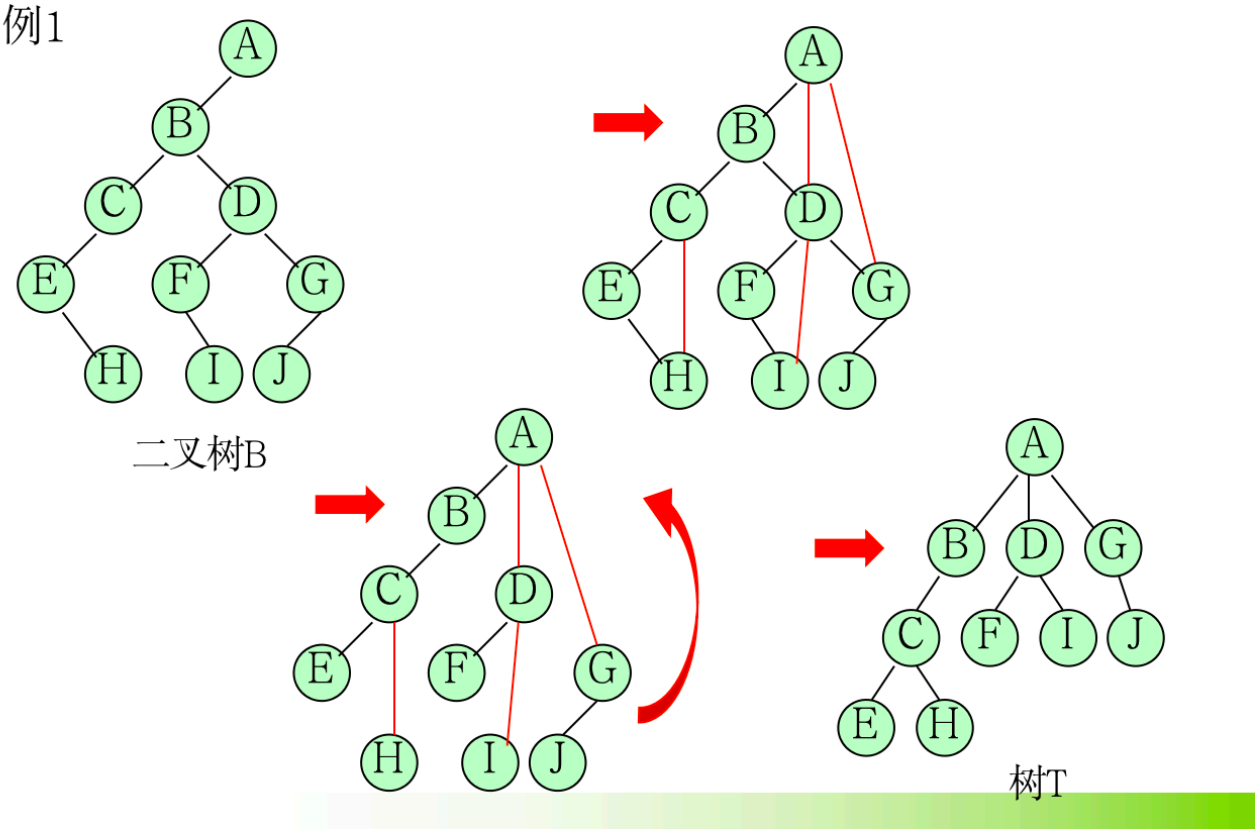
2.森林 ➡ 二叉树





3. 二叉树 ➡ 树

例1



6.6 哈夫曼树
相关概念：带权路径长度WPL
$WPL=\sum_{k=1}^{n} w_k l_k$

如何构造哈夫曼树，并给出每个字符的编码?
注意每一次归并之后都要排序哦

图

7.1 图的定义及相关术语
G=(V,E)
无向图、有向图、边、弧（弧头、弧尾）、完全图、有向完全图、网
子图、度、出度、入度、连通图、极大连通图、强连通图、极大强连通子图
生成树、有向树

- 若图G任意两顶点a,b之间的关系为有序对<a,b>,<a,b>∈E, 则称<a,b>为从a到b的一条弧/有向边；

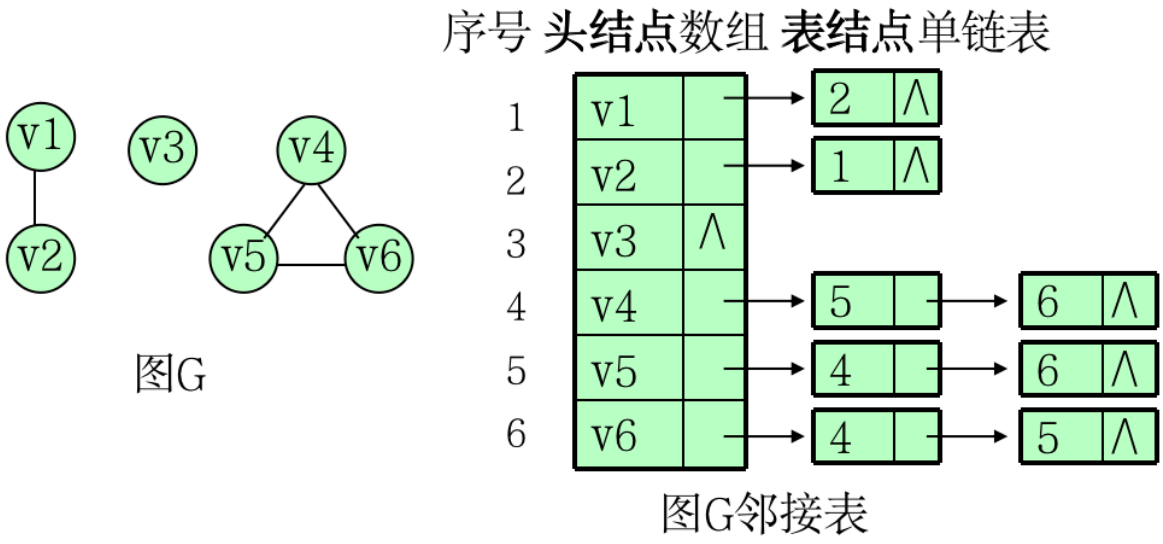
其中： a是<a,b>的弧尾，
b是<a,b>的弧头；
称该图G是有向图。

- 若图G的任意两顶点a,b之间的关系为无序对(a,b), 则称(a,b)为无向边(边）,称该图G是无向图。
- 网(Network),边(弧)上加权(weight)的图。
- 对图 G=(V,E)和G'=(V',E'),
若V'∈V 且 E'∈E，则称G'是G的一个子图
- 与顶点x相关联的边(x,y)的数目,称为x的度
- 以顶点x为弧尾的弧<x,y>的数目,称为x的出度，记作OD(x)。
- 以顶点x为弧头的弧<y,x>的数目,称为x的入度，记作ID(x)。
- 若从顶点v_i到v_j有路径,则称v_i和v_j是连通的。
- 若图G中任意两顶点是连通的,则称G是连通图。
- 若图G'是G的一个极大连通子图,则称G'是G的一个连通分量。
- 若图G'是G的一个极小连通子图,则称G'是G的一个生成树

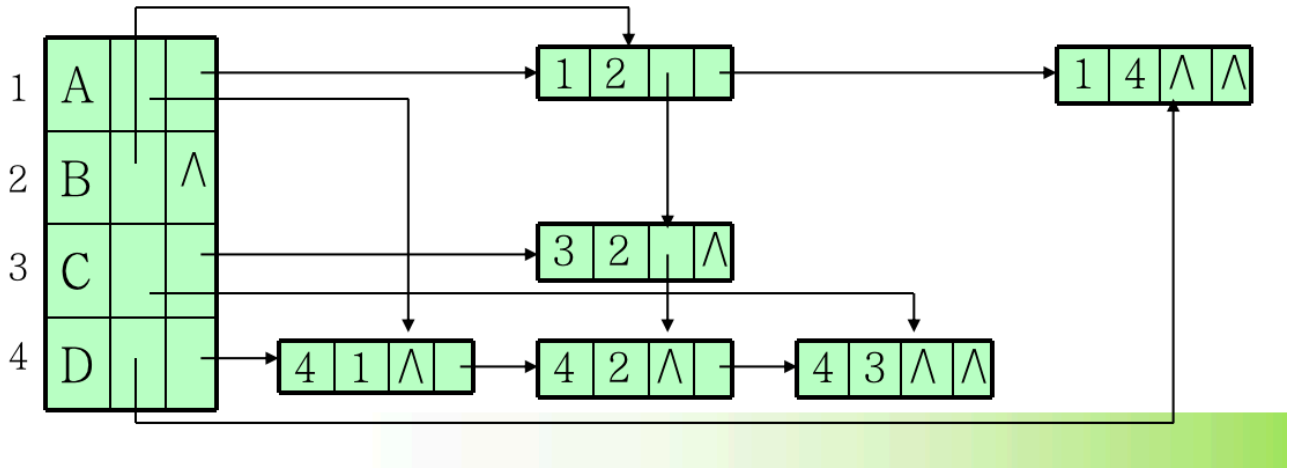
7.2 图的存储结构
数组表示法
邻接表、逆邻接表
十字链表
邻接多重表

邻接矩阵(略)

邻接表(做数据结构实验吐了嘛?)

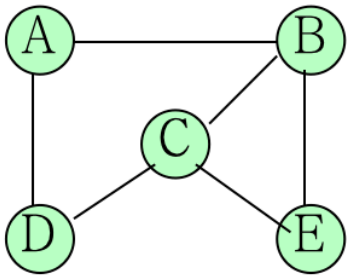


十字链表(把图看成一个稀疏矩阵存进去)
以邻接表为基础，扩展结点属性成起止结点序号,4个属性代表了什么?
1:代表指向的,2:代表被指向的

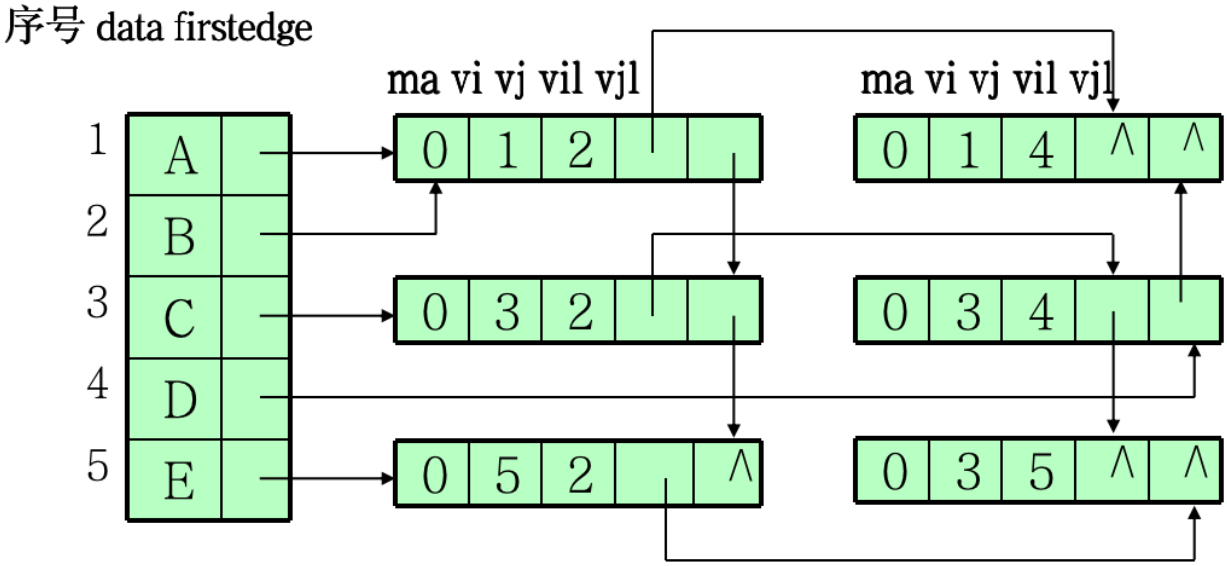


邻接多重表

图的一个顶点用一个“头结点”表示，
其中：
data域 存储和该顶点相关的信息，
firstedge域 存储第一条依附于该顶点的边。
图的一条边用一个“结点”表示，
其中： mark----标志域，可用以标记该条边是否被搜索过；
vi和vj----该条边依附的两个顶点在图中的位置；
vilink----指向下一条依附于顶点vi的边；
vjlink----指向下一条依附于顶点vj的边。
避免了无向图邻接表的一条边用两个结点。



G



7.3 图的遍历

深度优先搜索（DFS)、生成树、森林

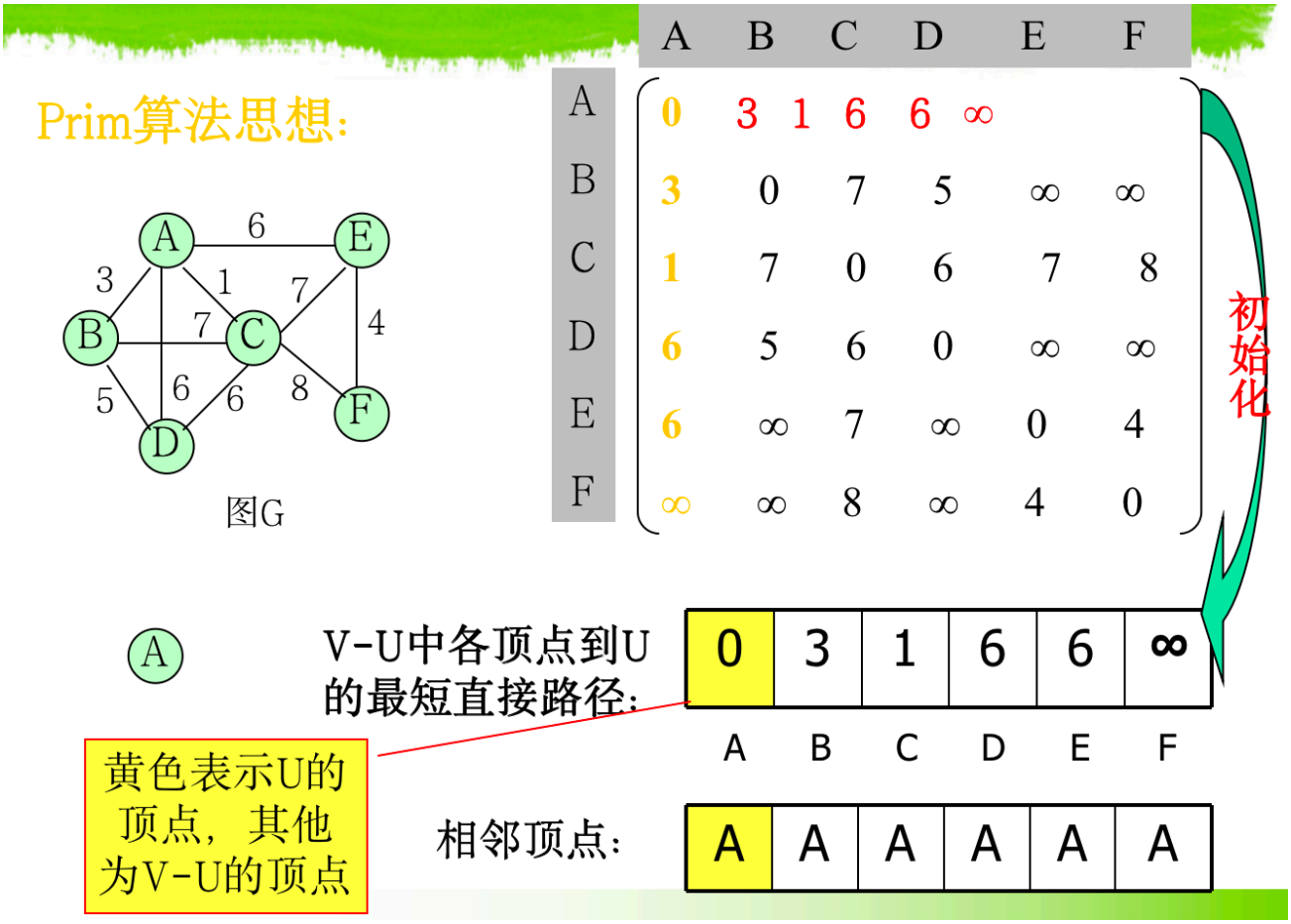
广度优先搜索（BFS)、生成树、森林

网的最小生成树：普里姆(prim)算法（以选顶点为主）

克鲁斯卡尔（Kruskai)算法（以选边为主）

适应的范围 稀疏矩阵和稠密矩阵

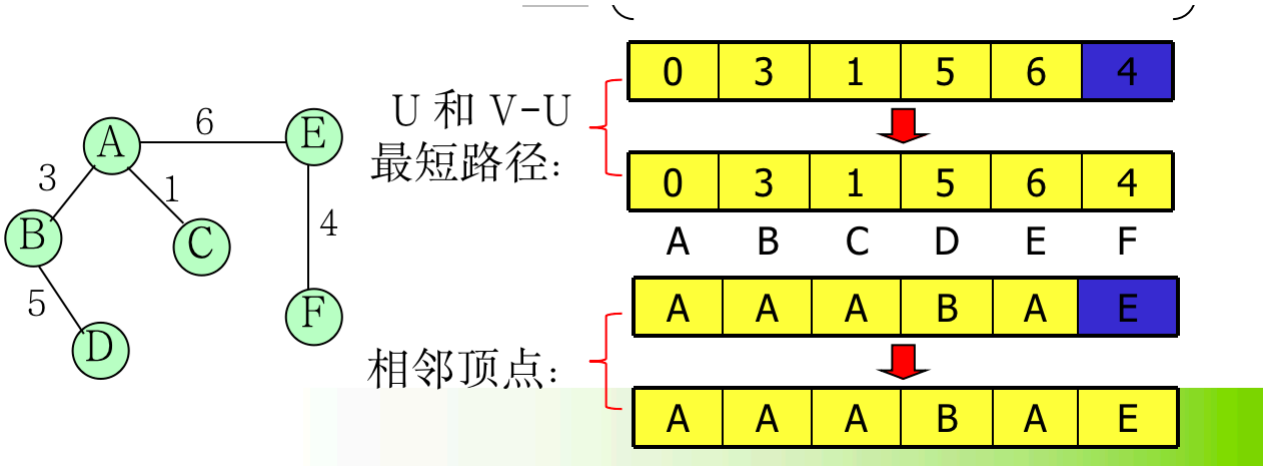
Prim算法的实现过程:规定从一个结点开始统计,初始化的过程:把A道每个结点的元素存进去



每一次循环都做什么?第一步:数组里没有被标成黄色的元素中距离最小的结点,标成黄色

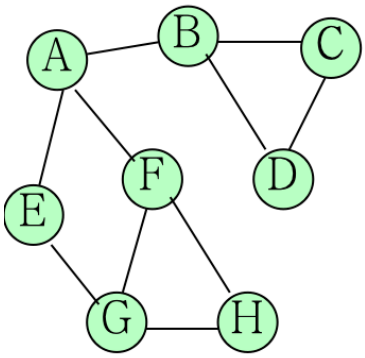
第二步:遍历一遍第一步结点所对应的矩阵的第一行,更新一下数组,如果某个元素的值比数组小,更新值以及其相邻结点

直到每个元素都是黄色了为止

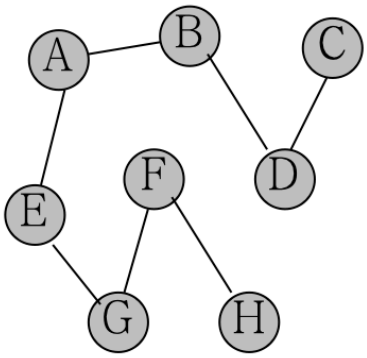


最后的相邻结点和值就可以推算出一颗二叉树

DFS---Depth First Search



图G



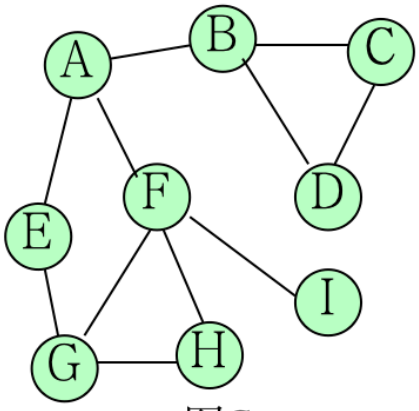
假定从 A 出发遍历图G:

- A,E,G,F,H,B,D,C
- A,B,D,C,E,G,H,F
- A,B,F,G,H,E,C,D
- A,E,F,H,G,B,C,D
- A,F,G,H,E,B,D,C

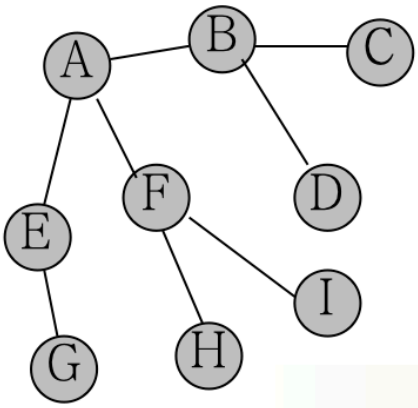
假定从G出发遍历图G:

- G,F,A,B,D,C,E,H
- G,H,F,A,E,B,D,C
- G,E,A,H,F,B,C,D

BFS---Breadth First Search



图G



假定从 A 出发遍历图G:

- A,E,F,B,G,H,I,D,C
- A,B,F,E,D,C,I,H,G
- A,F,E,G,H,I,B,D,C ?
- A,E,B,F,I,H,G,D,C ?

假定从 G 出发遍历G:

- G,F,E,H,A,I,B,C,D
- G,H,F,E,I,A,B,C,D
- G,E,F,H,I,A,B,C,D ?

最小生成树略

7.4 有向无环图（DAG图）

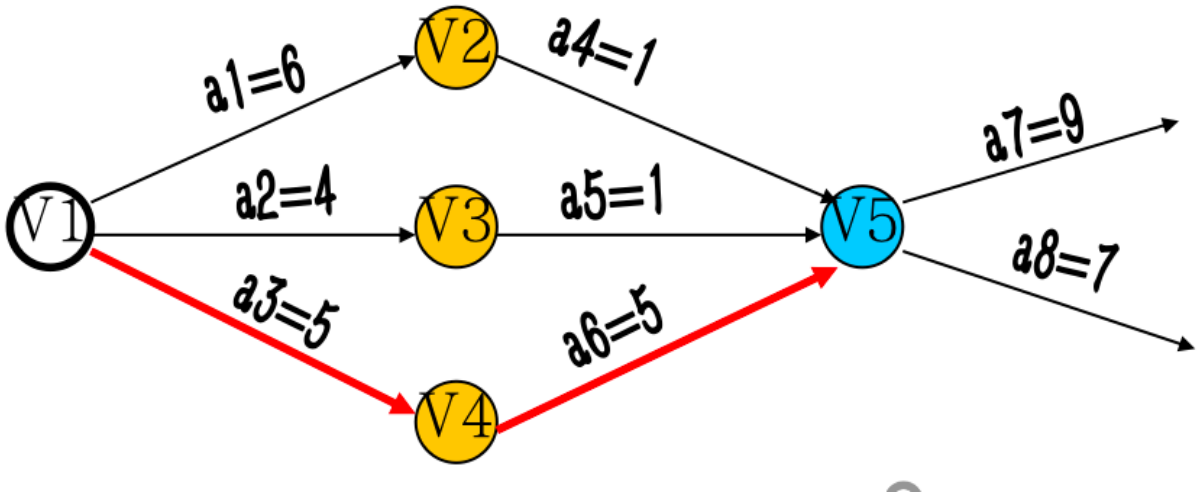
求拓扑排序的方法:每次找到入度为0的结点即可

求关键路径的方法:

求最短路径的方法:

从前往后->从后往前

1.(顶点）事件vi的最早发生时间ve(vi)：
有多个前驱顶点： ve=max{ve(前驱顶点)+前驱活动时间}

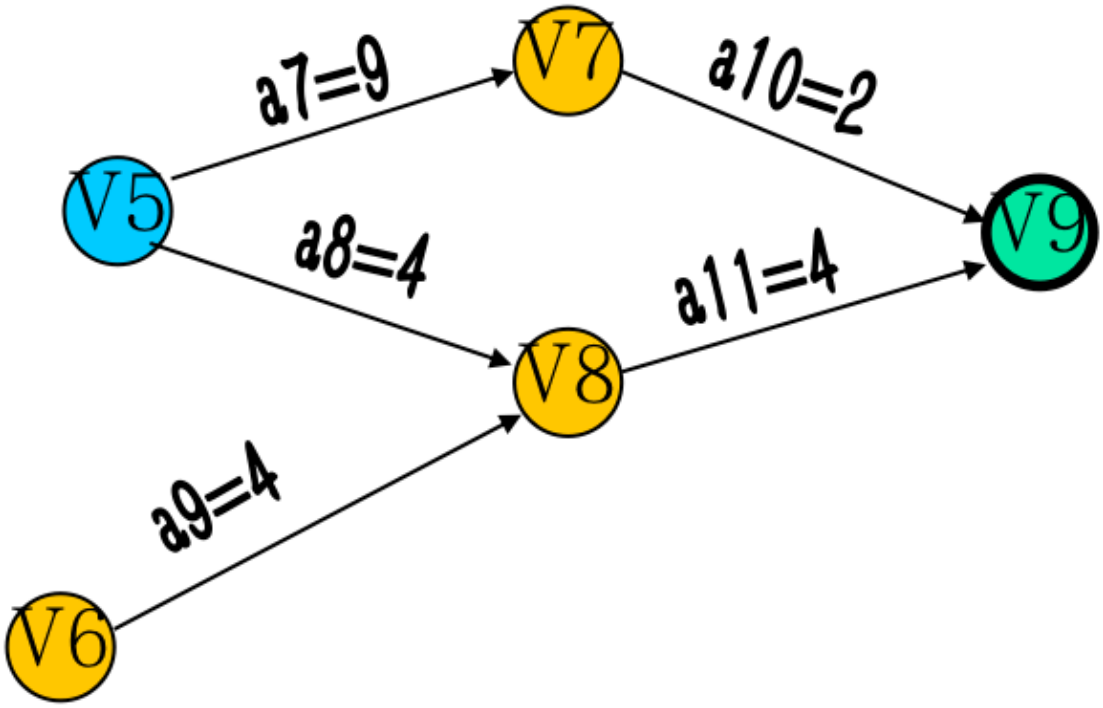


$v_e(v_5) = max\{6+1, 4+1, 5+5\} = 10$

2.既表示事件vi的最早发生时间，也表示所有以vi为尾的弧所表示的活动ak的最早发生时间$e(k)$

3.事件vi允许的最迟开始时间vl(vi) = 完成点（汇点）vn的的最早发生时间ve(vn)减去vk到vn的最长路径长度。

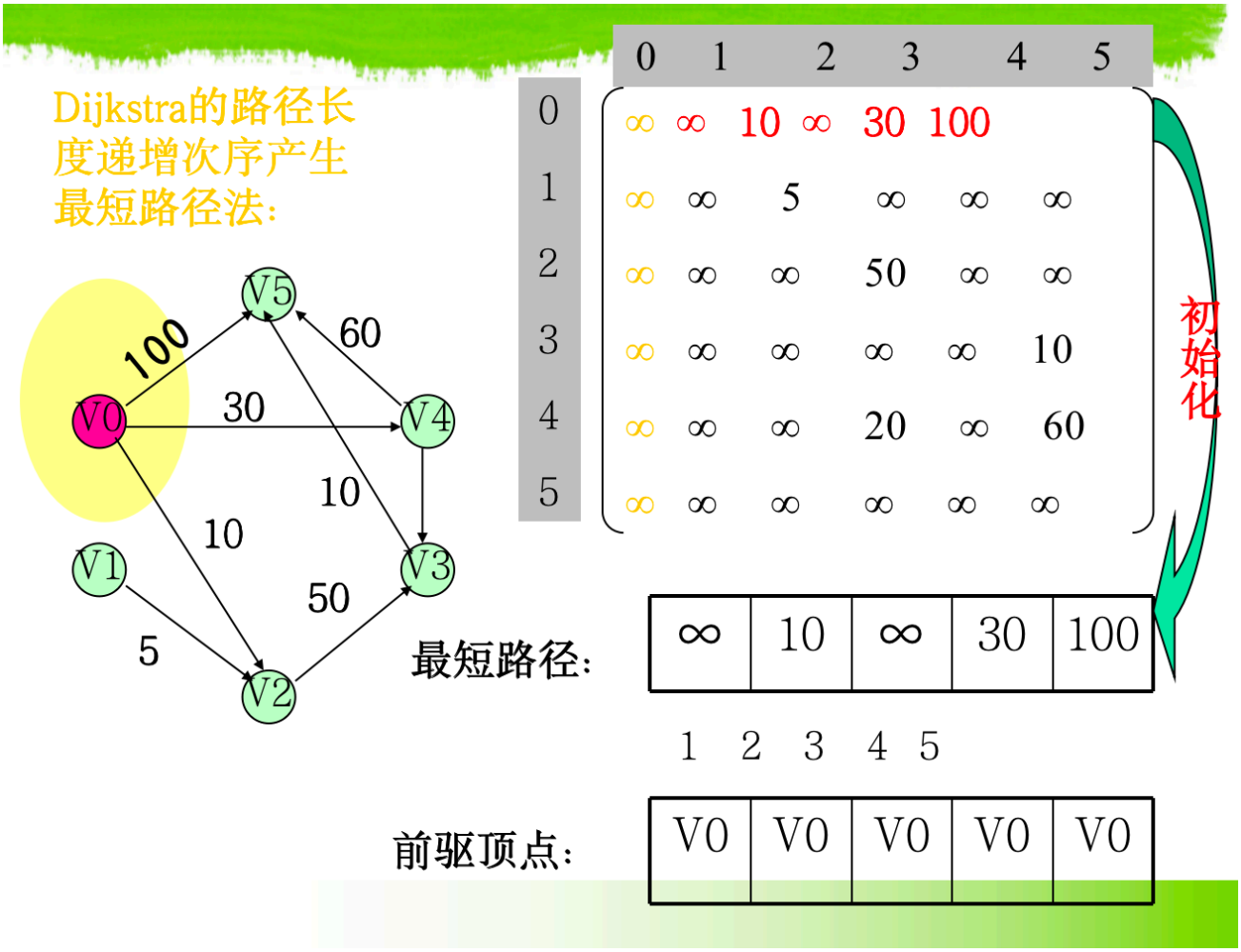
4.活动ak的最迟开始时间l(k)：l(ak)=vl(ak弧头对应顶点)-活动ak的持续时间



$v_l(v_5) = min\{v_l(v_7) - 9, v_l(v_8) - 4\} = min\{7, 10\} = 7$

7.5 最短路径

算法1（Dijkstra算法）：
以每一个顶点为源点，重复执行Dijkstra算法n次，即可求出每一对顶点之间的最短路径。



每一次:取出最小的元素,注意数组元素的更新从直接改变值到**上一个标黄的点元素值加上两点之间的距离**

算法2（Floyd算法）：
算法思想：
假设求Vi到Vj的最短路径，如果从Vi到Vj有弧，则存在一条长度为arcs[i][j]的路径，该路径不一定是最短路径，尚需进行n次试探。

查找

9.1 查找表的定义
关键字、主关键字、次关键字
静态查找、动态查找、平均查找长度（ASL）

- 关键字:　可以标识一个记录的数据项
- 主关键字: 可以唯一地标识一个记录的数据项
- 次关键字: 可以识别若干记录的数据项
- 查找----根据给定的某个关键字值，在查找表中确定一个其关键字等于给定值的记录或数据元素。
　　设k为给定的一个关键字值，R[1..n]为n个记录的表,若存在$R[i].key = k, 1 \le i \le n$,称查找成功；否则称查找失败。
- 静态查找: 查询某个特定的元素，检查某个特定的数据元素的属性,不插入新元素或删除元素(记录) 。
- 动态查找: 在查找过程中，同时插入查找表中不存在的数据元素(记录)。
- 平均查找长度----查找一个记录时比较关键字次数的平均值。

9.2 静态查找表
顺序表：顺序查找法，ASL
有序表：折半查找法、判定树，ASL
分块查找法、 ASL

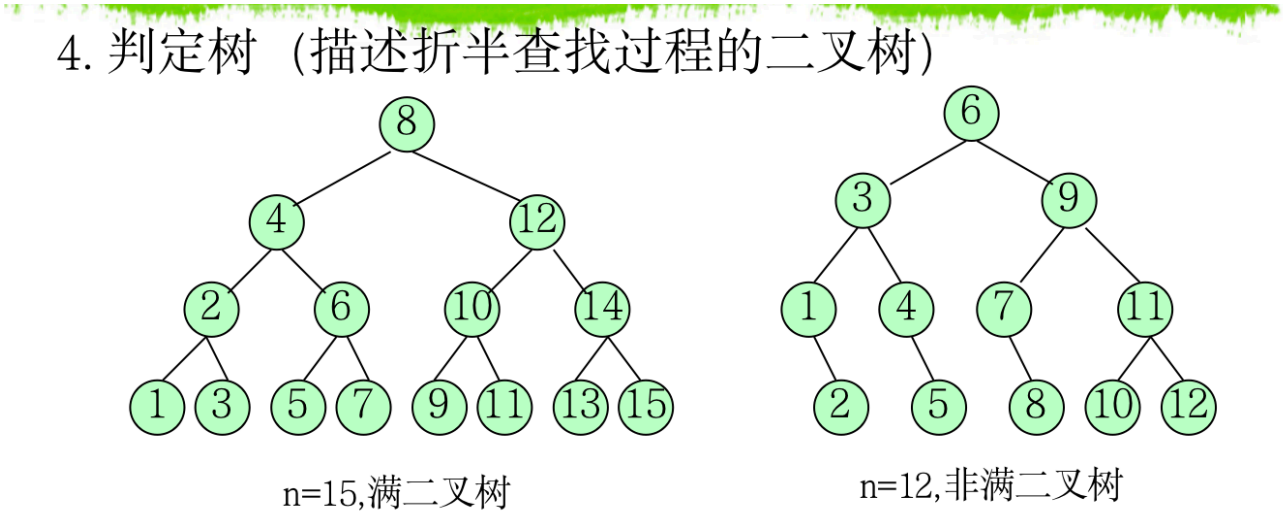
顺序表:就是线性表,注意哨兵的设置(哨兵就是把要查找的元素存进elems[0])
有无监视哨就是区别查找时有没有越界的判断,因为查找一般是从后往前做,有监视哨就可以保证就算没有在最后的elems[0]也能找到元素值

性能分析:只考虑查找成功$\frac{n+1}{2}$
考虑查找失败: 使用监视哨elem[0],为 n+1
不使用监视哨elem[0],为 n
综合下来$\frac{3(n+1)}{4}$

二分查找(C语言里讲过)

```
int binsrch(SSTable  ST,keytype k)
{int low,mid,hig;
 low=1;  hig=ST.length;
 while (low<=high)
 { mid=(low+high)/2;
   if (k<ST.elem [mid].key) hig=mid-1; //查左子表
   else if (k==ST.elem [mid].key)
          return mid;       //查找成功，返回mid
   else low=mid+1;    //查右子表
 }
 return 0 ;              //查找失败，返回0
}
```

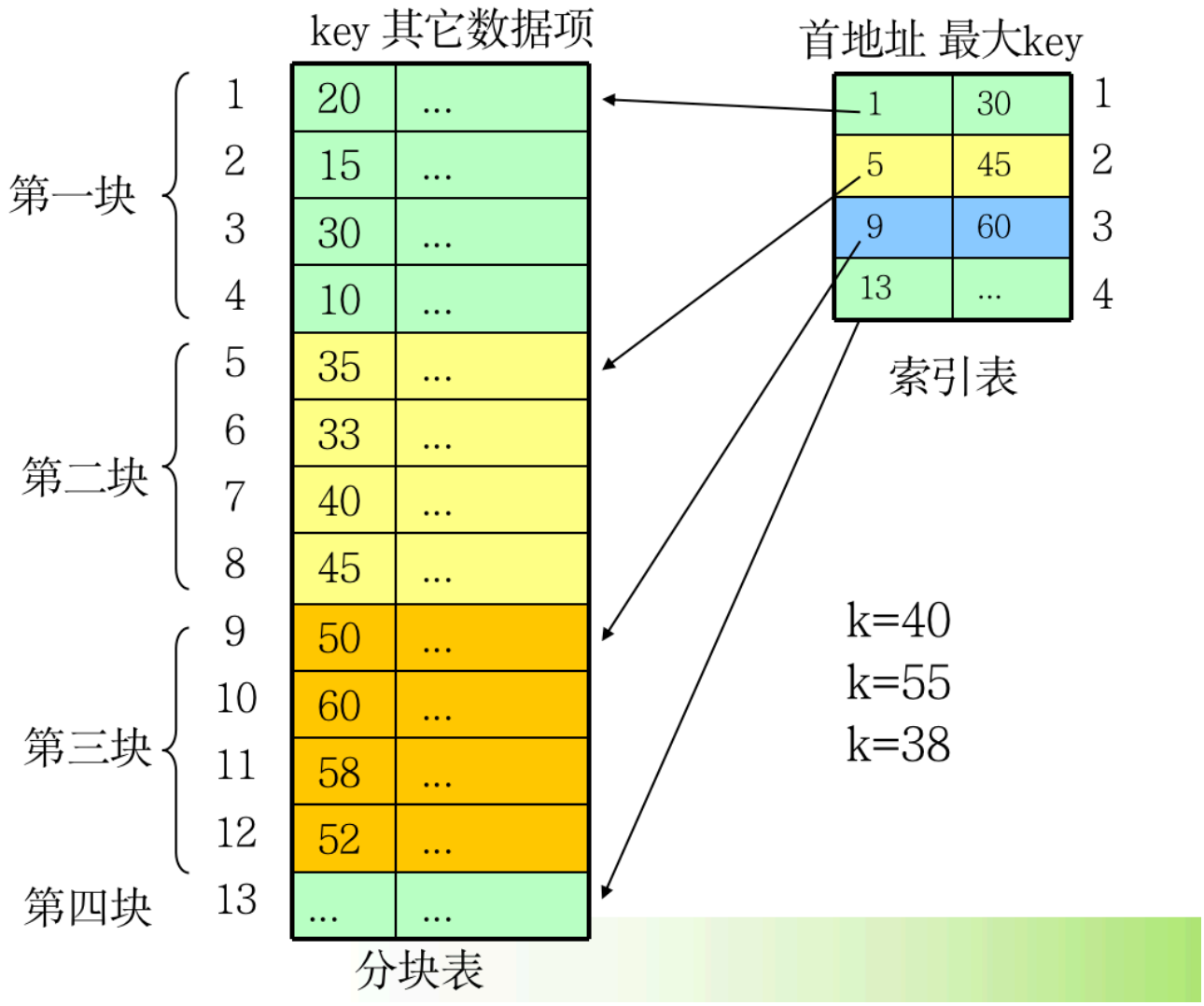
判定树怎么画?
第一个查找到的元素放第一层,第二个查找到的元素放在第二层,以此类推.....



➢结点内的数据表示数据元素的序号（如例中表示有15个元素组成的有序表的判定树）
➢根结点表示首先要和关键字k进行比较的数据元素的序号（如8），比较相等时，查找成功，否则，当k小于根结点对应元素的关键字时，下步就和左子结点（如序号4）对应元素的关键字比较，否则，下步就和右子结点（如序号12）对应元素的关键字比较。

对于任意的n,ASL都可以表示成$O(log_2 n)$之形式

分块索引



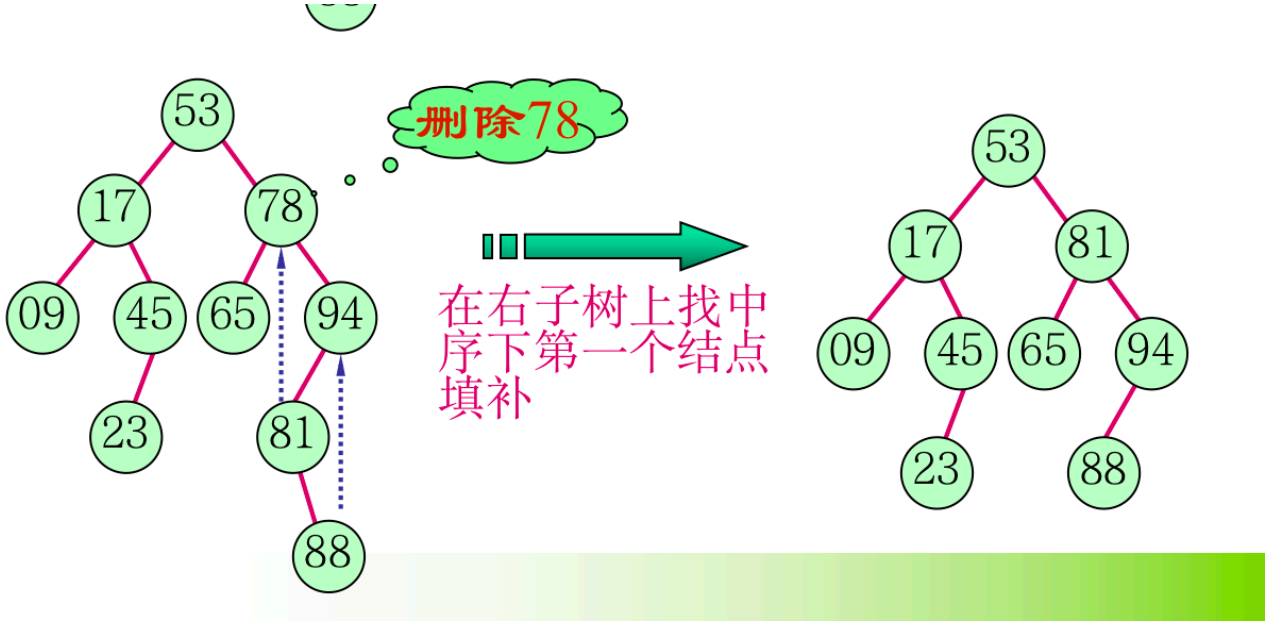
● 条件
(1)分块表"按块有序", 索引表"按key有序"
(2)设n个记录分为b个块，每块的记录数s=n/b
●查找方法:先找块,再进入块里寻找
●ASL:最佳分块:$s=\sqrt{n}, d=\sqrt{n}, ASL=O(\sqrt{n})$

9.3 动态查找表
二叉排序树的生成
插入、删除元素到二叉排序树的算法
二叉排序树的查找算法、ASL
平衡二叉树

●如果二叉树的任一结点大于其非空左子树的所有结点，而小于其非空右子树的所有结点，则这棵二叉树称为二叉排序树。对一棵二叉排序树进行中序遍历，所得的结点序列一定是递增有序的。
●生成二叉树:做n次二叉树排序
●存储结构

```
struct  node
  { struct
    { int key ;                 //关键字
      .....                     //其它数据项
     } data ;
   struct node *lchild,*rchild ; //左右子树的指针
   } *root,*t;
```

●插入:大的话就到右边,小的话到左边,遇到空就插入
●查找:找到了就算成功,找不到就返回NULL,跟插入一个样子
●删除:删除叶结点，只需将其双亲结点指向它的指针置空，再释放它即可。
被删结点缺左子树（或右子树），可以用被删节点的右子树（或左子树）顶替它的位置，再释放它。
被删结点左、右子树都存在，可以在它的**右子树中寻找中序下的第一个结点(关键值最小,或者说是最左边的元素)**,用它的值填补到被删结点中，再来处理这个结点的删除问题。(往往这个结点没有左孩子,就可以降级到第二个问题了)
ASL:
平均情况:$O(log_2n)$
最坏情况:$\frac{n+1}{2}$

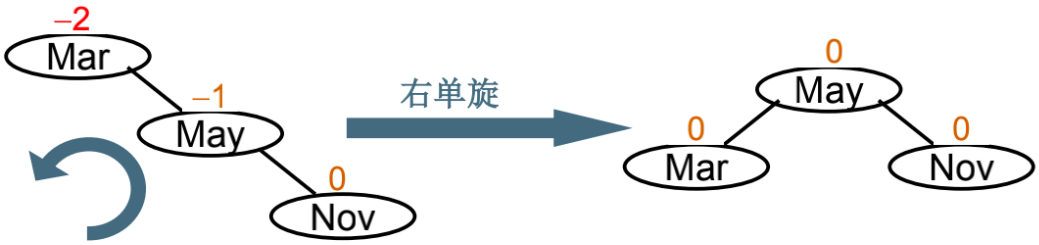


●AVL树：由G.M.Adelson-Velskii和E.M.Landis提出。
●结点的平衡因子：结点的左右子树的深度之差。
●平衡二叉树：任意结点的平衡因子的绝对值
　　小于等于1的二叉树。

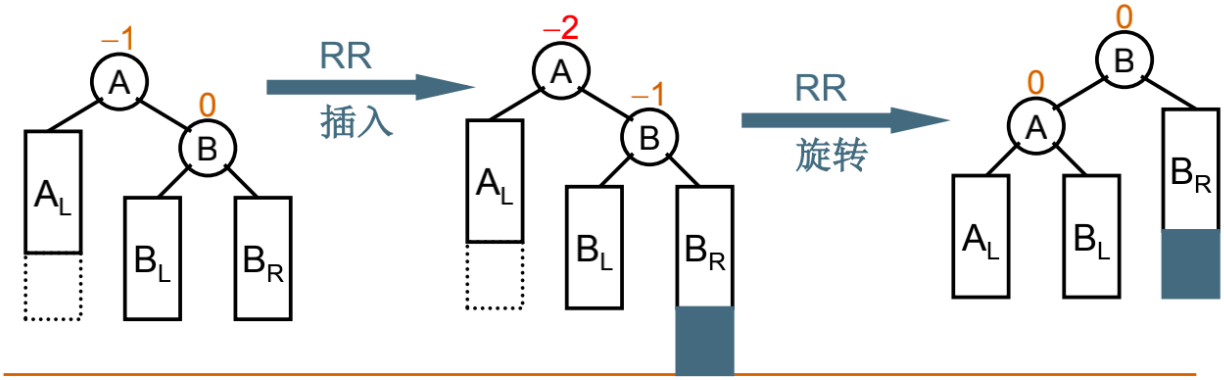
●如果在一棵平衡的二叉搜索树中插入一个新结点，造成了不平衡。此时必须调整树的结构，使之平衡化。
平衡化旋转有两类：
单旋转 (左旋和右旋)
双旋转 (左平衡和右平衡)
●如果这三个结点处于一条直线上，则采用单旋转进行平衡化。单旋转可按其方向分为左单旋转和右单旋转，其中一个是另一 个的镜像，其方向与不平衡的形状相关。
如果这三个结点处于一条折线上，则采用双旋转进行平衡化。双旋转分为先左后右和先右后左两类。

平衡二叉树的调整

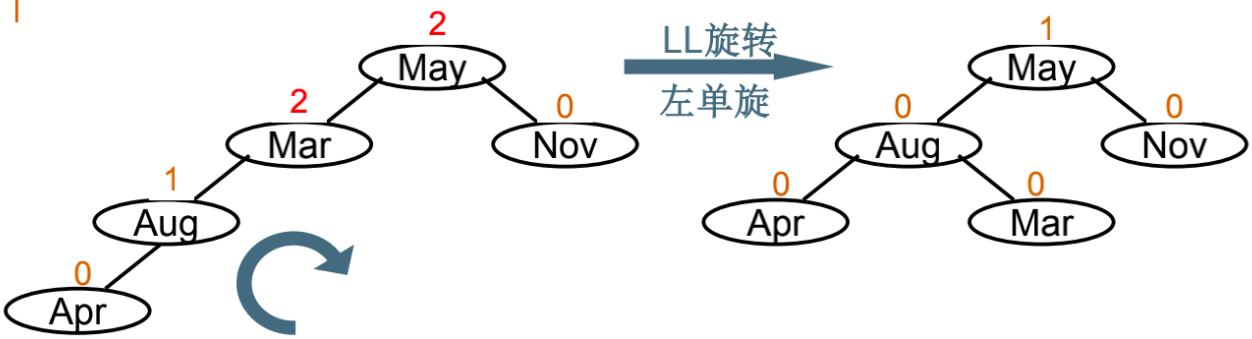




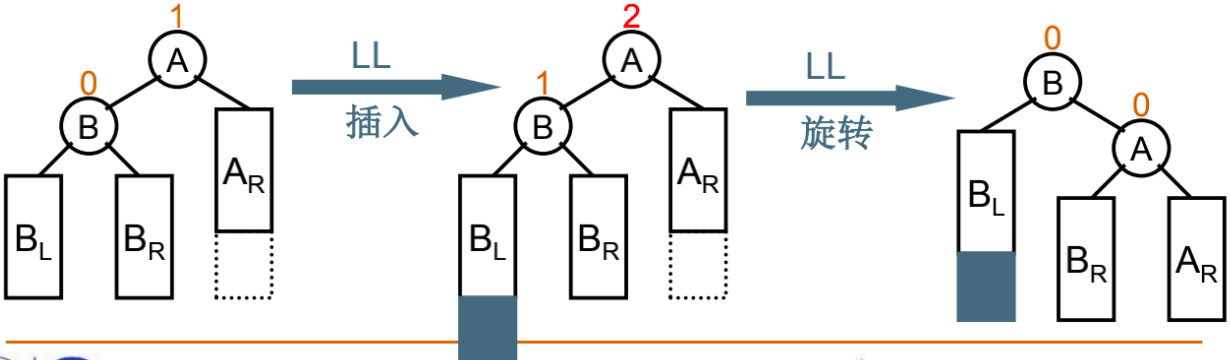
不平衡的“发现者”是**Mar**，“麻烦结点”**Nov** 在发现者右子树的右边，因而叫 **RR** 插入，需要**RR** 旋转（右单旋）

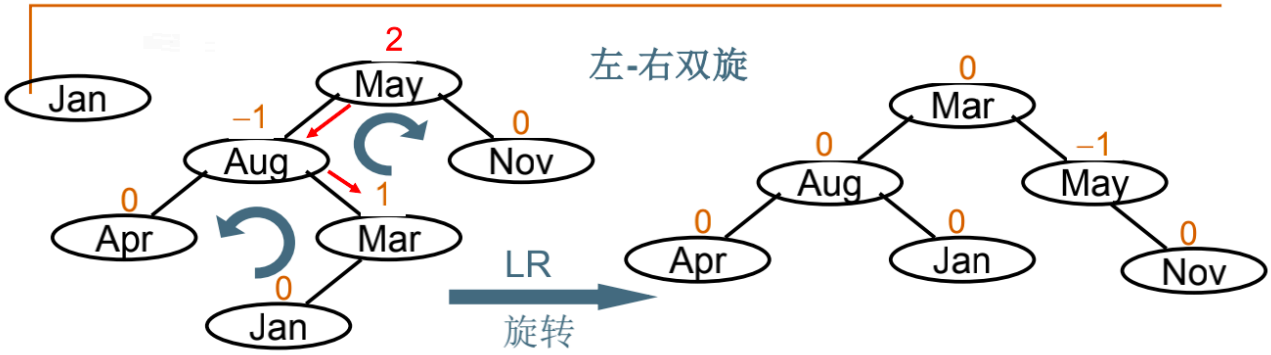




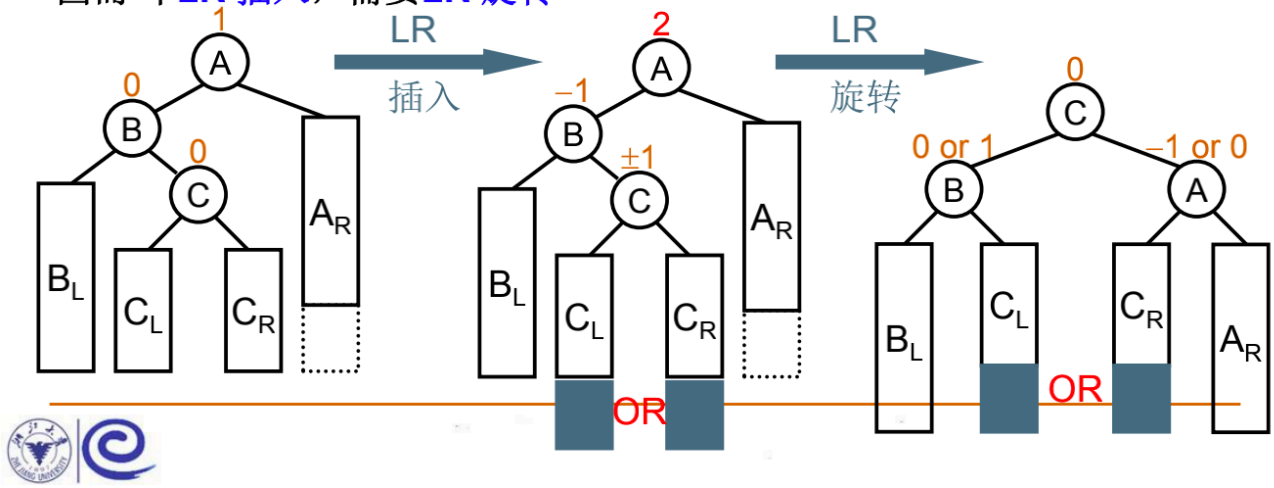


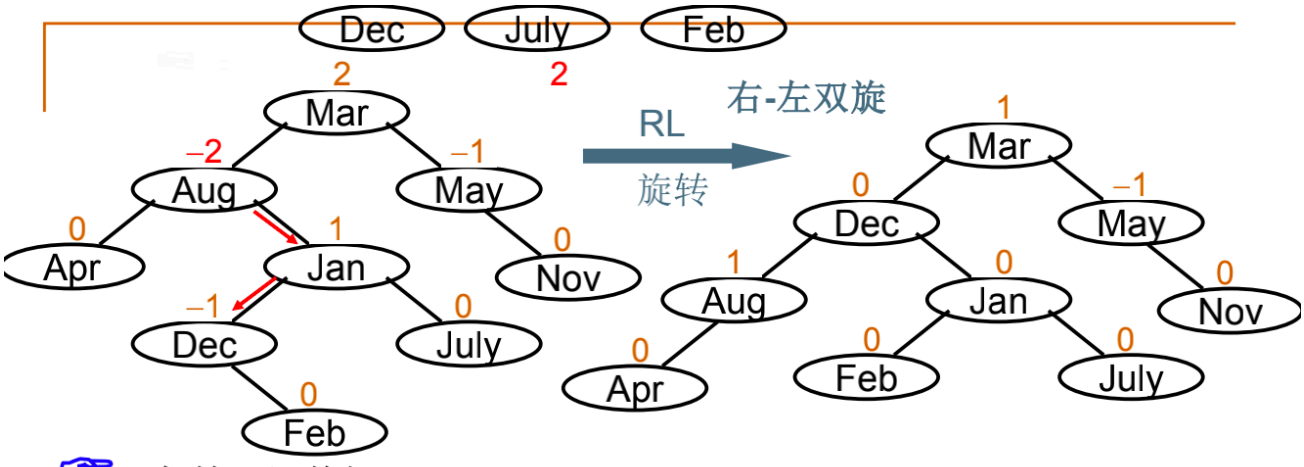
“发现者”是**Mar**，“麻烦结点”**Apr** 在发现者左子树的左边，因而叫 **LL** 插入，需要**LL** 旋转（左单旋）



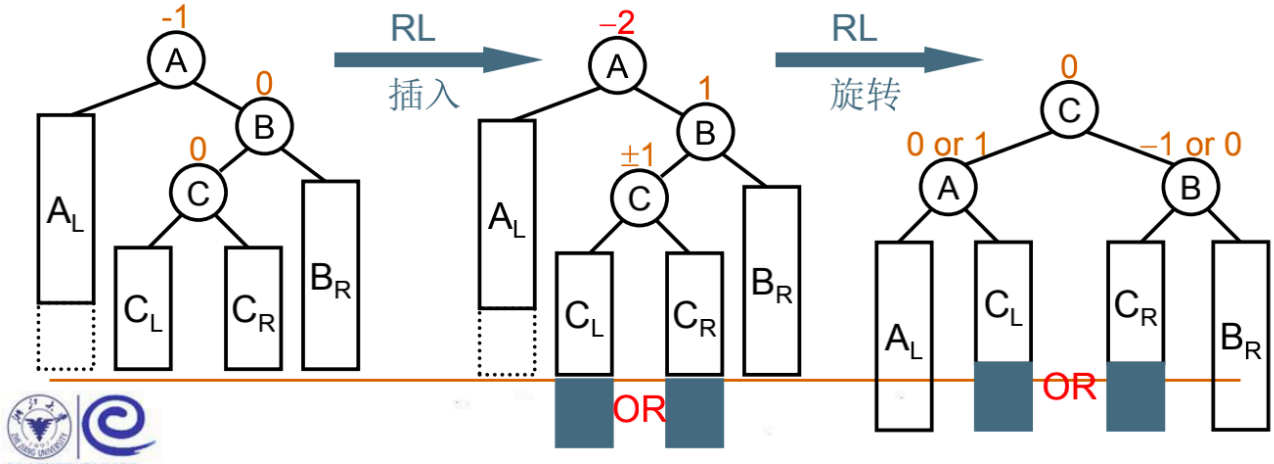


☞ “发现者”是**May**，“麻烦结点”**Jan**在左子树的右边，
因而叫 **LR 插入**，需要**LR 旋转**





☞ 一般情况调整如下:



9.4 哈希表
构造哈希函数的方法
处理冲突的方法（线性探测再散列、二次探测再散列、链地址法）
如何构造哈希表、处理冲突、计算ASL（查找成功和查找失败）
装填因子

●Hash函数实现的是将一组关键字映象到一个有限的地址区间上，理想的情况是不同的关键字得到不同的地址。
设K1和K2为关键字, 若K1≠K2,H(K1)=H(K2),则称K1,K2为同义词,K2与K1发生了冲突

●构造哈希函数的方法
1.直接定址法
2.除留余数法:设哈希表HT[0..m-1]的表长为m，哈希地址为key除以p所的的余数：
3.平方取中法----取关键字平方后的中间某几位为哈希地址,即:H(k)=取k2的中间某几位数字
4.折叠法
　将关键字分割成位数相同的几部分,然后取这几部分的叠加和作为哈希地址。
5.数字分析法
　设哈希表中可能出现的关键字都是事先知道的,则可取关键字的若干分布均匀的位组成哈希地址。
6.随机数
7.*灵活构造哈希函数*

●解决冲突的方法
1.开放地址法(开式寻址法)
线性探测再散列:这个不行就找下一个地址,直到找到了元素的地址,轮了一圈没找着那就不行,报错
二次探测再散列
　假定记录Ri和Rj的关键字Ki和Kj为同义词，散列地址为q, R_i已存入HT[0..m-1]中的HT[q]中。若依次在地址$q+1^2, q-1^2, q+2^2, q-2^2, \ldots, q+i^2, q-i^2$,...中寻找一个空位，叫做二次探测再散列。
2.链地址法
　将关键字为同义词的所有记录存入同一链表中。(表尾插入)
3.建立公共溢出区
4.再哈希

●ASL的计算
成功的ASL:每一个元素寻找的次数/总元素数
失败的ASL:对应每个位置寻找的次数/总元素数(因为再探测时还是没找到那就是根本就没有了)

●装填因子:α=(表中填入记录数)/(哈希表的长度)

排序

10.1 排序的概念
时间复杂度:比较关键字的次数、移动记录的次数
空间复杂度
排序的稳定性:存在两个具有相同关键字的记
录R(i)与R(j),其中R(i)位于R(j)之前。在用某种排序法排序
之后,R(i)仍位于R(j)之前，则称这种排序方法是稳定的；否则，称这种排序方法是不稳定的。
10.2 直接插入排序
直接插入算法、比较关键字的次数、移动记录次数,稳定性

设待排序的文件为：(r[1],r[2],...,r[n])
　关键字为：(r[1].key,r[2].key,...,r[n].key)
　首先,将初始文件中的记录r[1]看作有序子文件；　　第1遍：将r[2]插入有序子文件中，若：
　　r[2].key<r[1].key,
　则r[2]插在r[1]之前；否则，插在r[1]的后面。　　第2遍：将记录r[3]插入前面已有2个记录的有序子文件中，得到3个记录的有序子文件。
　以此类推，依次插入r[4],...,r[n],最后得到n个记录的递增有序文件。

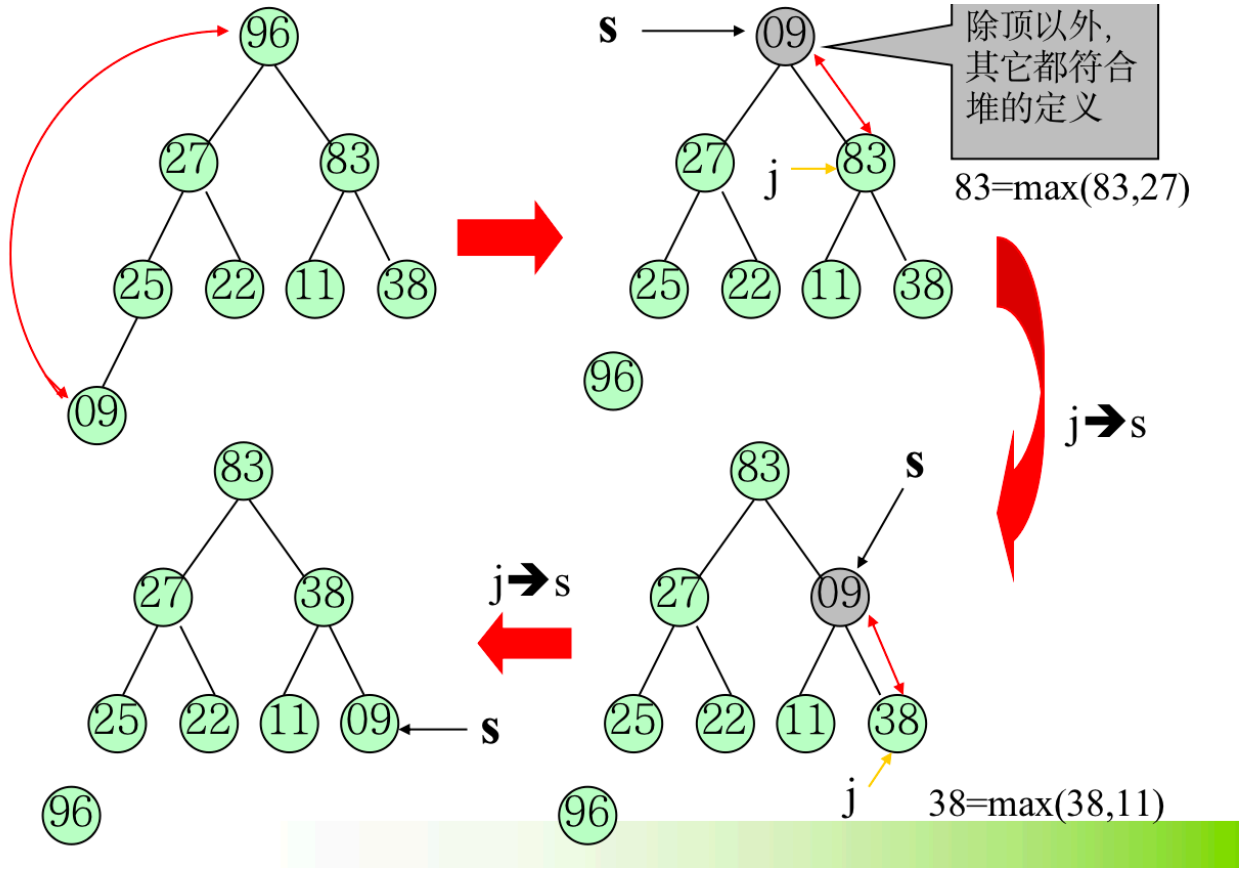
```
void InsertSort(RecType r[],int n)
  // 对数组r[1..n]中的n个记录作插入排序
{ int i,j;
  for (i=2; i<=n; i++){
     r[0]=r[i];              //待插记录r[i]存入监视哨中
     j=i-1;                  //以排序的范围1 — i-1
                             //从r[i-1]开始向左扫描
     while(r[0].key<r[j].key)
     { r[j+1]=r[j];          //记录后移
       j--;                  //继续向左扫描
      }
     r[j+1]=r[0];            //插入记录r[0],即原r[i]
    }
  }
```

10.3 选择排序
简单排序算法、比较关键字的次数、移动记录次数，稳定性
堆排序算法、大顶堆、小顶堆、如何建立初始堆、替换后调整新堆的方法、算法分析

简单排序算法:一个一个选出最小的,插进去

```
void SelectSort(RecType r[],int n)
{ int i,j,min;
  RecType x;                  //交换记录的中间变量
  for (i=1;i<n;i++)           //共n-1趟(遍)
   { min=i;                   //r[i]为最小记录r[min]
     for (j=i+1;j<=n;j++)
       if (r[j].key<r[min].key)
         min=j;               //修改min
     if (min!=i)              //若r[min]不是r[i]
      { x=r[min];             //交换r[min]和r[i]
        r[min]=r[i];
    r[i]=x;
       }
    }
 }
```

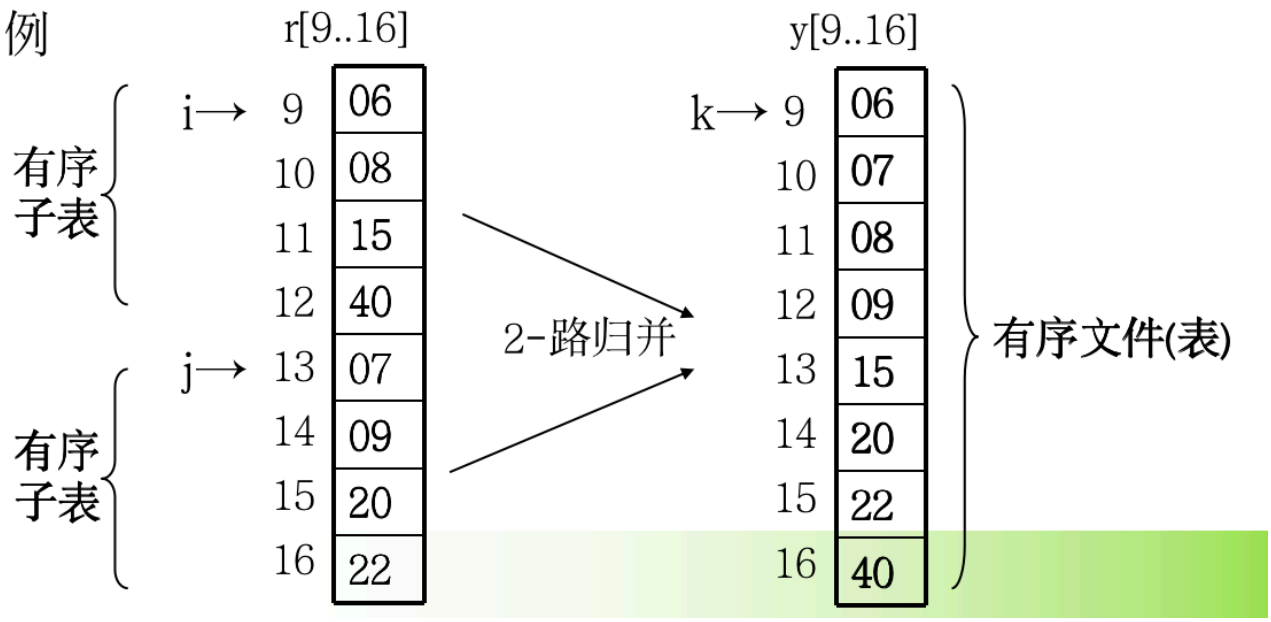
堆的调整:




```
void  HeapAdjust(HeapType  &H, int s, int  m)
{ rc=H.r[s];              //保存需调整的数据元素，空出s的记录位置
  for(j=2*s; j<=m;j*=2) {  //j<=m表示s有左孩子序号    j=2*s
    if (j<m&&H.r[j].key<H.r[j+1].key) //j<m表示s有右孩子j+1
         j++;               //计算s的具有较大关键字的孩子的序号j
    if (rc.key> H.r[j].key)//rc在s的结点时满足结点定义,调整完毕
         break;
      H.r[s]=H.r[j];s=j;     //s的较大孩子上移，修改s下移
     }                        //正常结束时，  s的结点无孩子结点。
    H.r[s]=rc;
}
```

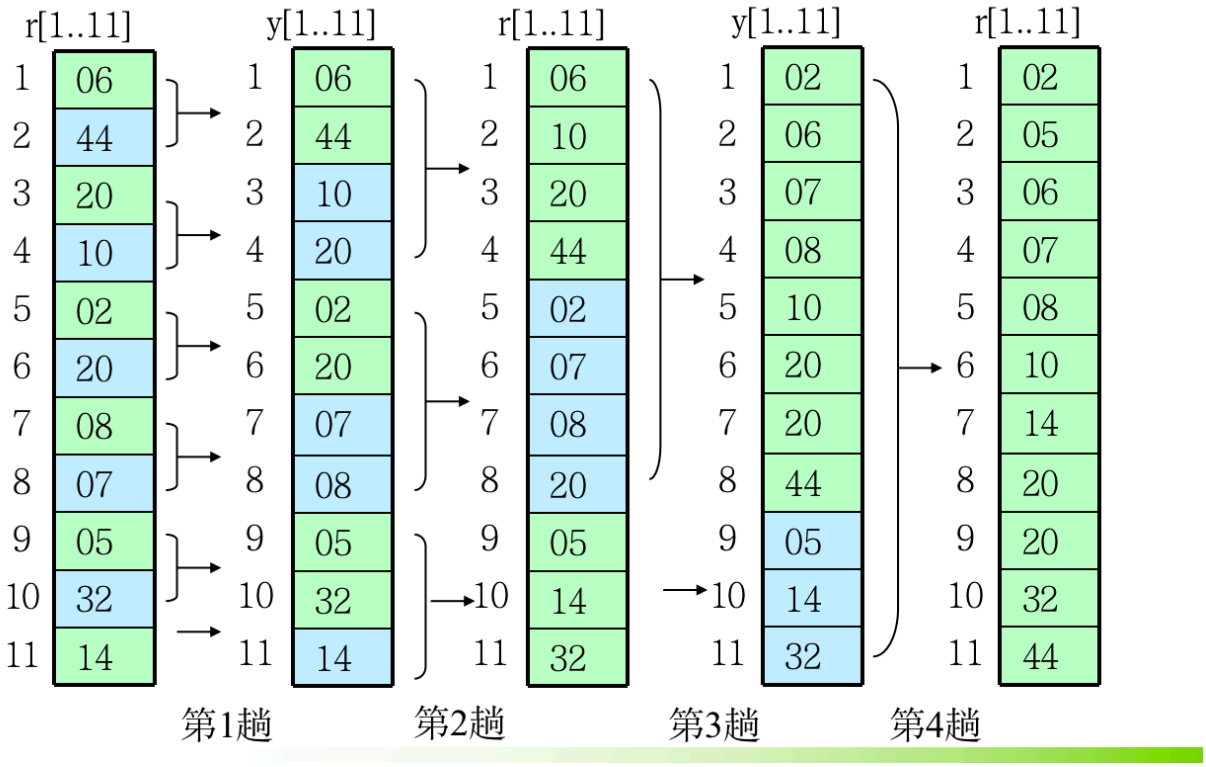
调整方法:找孩子中最大的,交换,一直到比孩子大或者越界
怎么初始化一个堆?从n/2开始调用堆调整的代码

10.4 2-路归并排序
算法思想，算法分析，稳定性




```
void merge(r,y,low,mid,high)
RecType r[],y[]; int low,mid,high;
{ int  k=i=low,j=mid+1;
  while (i<=mid && j<=high)
  { if (r[i].key<=r[j].key)
     { y[k]=r[i];     //归并前一个子文件的记录
       i++; }
    else
     { y[k]=r[j];     //归并后一个子文件的记录
       j++; }
    k++;
   }
     while (j<=high)    //归并后一个子文件余下的记录
   { y[k]=r[j];
     j++;  k++;
    }
     while (i<=mid)   //归并前一个子文件余下的记录
  { y[k]=r[i];
    i++;  k++;
   }
 } // merge
```

例2. 对11个记录作2-路归并排序,进行$\lceil \log_2 11 \rceil = 4$趟归并。



两个两个归并,剩下的元素数量不是整数倍怎么办?

正常情况下,剩下的元素的数量大于2s,那就归并

如果大于s小于2s,剩下一个正常的和残疾人归并

小于s,之间往下copy

```
//一趟归并
void mergepass(RecType r[], RecType y[],int s)
 { int i=1;
   while(i+2*s-1<=n)          //两两归并长度均为s的子文件
    { merge(r,y,i,i+s-1,i+2*s-1);
      i=i+2*s;
     }
   if (i+s-1<n)      //最后两个子长度为s和长度不足s的文件
     merge(r,y,i,i+s-1,n);
   else
     while(i<=n)              //复制最后一个子文件,长度≤s
      { y[i]=r[i];
        i++;
       }
   }
//对文件r[1..n]归并排序的算法(调用算法mergepass)
void mergesort(RecType r[],int n)
 {
   RecType y[n+1] ;
   int s=1;                  //子文件初始长度为1
   while (s<n)
    { mergepass(r,y,s);    //将r[1..n]归并到y[1..n]
      s=2*s;               //修改子文件长度
```

```
      mergepass(y,r,s);   //将y[1..n]归并到r[1..n]
      s=2*s;              //修改子文件长度
    }
  }
```

10.5 交换排序
冒泡排序算法，算法分析，稳定性
快速排序算法、算法分析，稳定性

```
void  bubble1(int a[],int n)
  { int i,j,temp;
    for(i=0; i<n-1; i++)      //作n-1趟排序
      for(j=0; j<n-1-i; j++)
        if (a[j]>a[j+1])
          { temp=a[j];        //交换记录
            a[j]=a[j+1];
            a[j+1]=temp;
           }
    for(i=0; i<n; i++)
      printf("%d",a[i]);   //输出排序后的元素
   }
//可以加一个swap来判断是不是完全有序了
```

快速排序之思想:

先选择第一个元素放到最前,保存到临时变量x,然后往前遍历,比x低的放到前面,比x大的放到后面,通过移动i和j指针进行处理

```
void quksort(RecType r[],int low,int high)
{  RecType x; int i,j;
  if (low<high)                          //有两个以上记录
   { i=low; j=high; x=r[i];             //保存记录到变量x中
     do {                                //此时i指示位置可用
         while (i<j && r[j].key>=x.key)
           j--;      //j从右向左端扫描通过key不小于x.key的元素
         if (i<j)                           //i,j未相遇
          { r[i]=r[j];  i++;               //此时j指示位置可用
            while(i<j && r[i].key<=x.key)
              i++;    //i从左向右端扫描通过key不大于x.key的元素
           if (i<j)
             { r[j]=r[i]; j--; }
          }
        } while (i!=j);                     //i,j未相遇
 }
    //划分结束，i经过的是key不大于x.key的元素；
                j经过的是key不小于x.key的元素。
                i,j至少有一个指示的位置可用
      r[i]=x;
      quksort(r,low,i-1);          //递归处理左子文件
      quksort(r,i+1,high);         //递归处理右子文件
     }
  }
```

对文件r[1..n]快速排序:

```
void quicksort(RecType r[],int n)
{
 quksort(r,1,n);
}
```

10.6 基数排序
算法思想，算法分析，稳定性

（1）首先根据最高有效位进行排序，然后根据次高有效位进行排序，直到根据最低有效位进行排序，产生一个有序序列。
（2）首先根据最低有效位进行排序，然后根据次低有效位进行排序，直到根据最高有效位进行排序，产生一个有序序列。

288,371,260,531,287,235,56,299,18,23。
第一趟排序后，结果为：260,371,531,23,235,56,287,288,18,299
第二趟排序后，结果为：18,23,531,235,56,260,371,287,288,299
第三趟排序后，结果为：18,23,56,235,260,287,288,299,371,531
两趟排序之间不改变元素的之间的相对位置,上一轮排序落后的,下一轮排序数值一样还是在后面

排序方法	平均时间复杂度	最坏情况下时间复杂度	额外空间复杂度	稳定性
简单选择排序	$O(N^2)$	$O(N^2)$	O(1)	不稳定
冒泡排序	$O(N^2)$	$O(N^2)$	O(1)	稳定
直接插入排序	$O(N^2)$	$O(N^2)$	O(1)	稳定
希尔排序	$O(N^d)$	$O(N^2)$	O(1)	不稳定
堆排序	O(NlogN)	O(NlogN)	O(1)	不稳定
快速排序	O(NlogN)	$O(N^2)$	O(logN)	不稳定
归并排序	O(NlogN)	O(NlogN)	O(N)	稳定
基数排序	O(P(N+B))	O(P(N+B))	O(N+B)	稳定